FABRIC FACTORY

Fabric Factory and the evpn-3tier Lab

A self-service, intent-driven EVPN/VXLAN data centre: architecture, tech stack, and the automation and orchestration that take a request from a web form to a verified change on 20 virtual devices.

DOCUMENT	Technical white paper
LAB VERSION	evpn-3tier 0.18.0
DATE	9 November 2026
SCOPE	Network design, platform, application, sources of truth, CI/CD pipelines, the self-service app, resilience and operations

Contents

1. Executive summary
2. Platform and deployment
3. The network fabric
4. Routing and security policy
5. The application
6. Sources of truth and what is in Nautobot
7. Automation and orchestration
8. Fabric Factory, the self-service app
9. A change end to end: Add a VLAN
10. Resilience and verification
11. The tech stack
12. Operating the lab
13. Design decisions and lessons learned
14. Limitations and roadmap

1 Executive summary

evpn-3tier is a complete data centre in software: a VyOS EVPN/VXLAN fabric, a stateful firewall pair, a WAN router and a three-tier web application, running as 20 virtual machines on one host. Fabric Factory, its self-service front end, turns a request typed into a web form into a verified network change: an operator approves, and the automation reserves addresses in phpIPAM, records intent in Nautobot, configures the devices through four Ansible pipelines in Gitea, and proves the result by pinging every server from the WAN.

20 VMs: 11 VyOS, 9 Alpine Linux	**4** VRFs; the firewalls are the only path between them	**4** Gitea pipelines, one per device group, five steps each	**65** end-to-end checks in `verify`; 9 test suites

Four ideas shape the lab:

- **Intent, not commands.** Nobody configures a device by hand. phpIPAM holds the addressing, Nautobot holds the plan, and pipelines render every configuration from the plan. Deleting intent deletes configuration.
- **Isolation by routing.** Each application tier lives in its own VRF and only learns a default route that points at the firewalls, so every packet between tiers, or to the WAN, crosses the zone firewall.
- **Fewer fields, fewer mistakes.** A requester never types a VLAN ID, a VNI, a name, a subnet or a gateway. When a VLAN is requested, the system fills those in itself: the next unused VLAN ID and the next free subnet, from the sources of truth. By design, there is nothing to mistype and nothing to collide.
- **Visible, reversible automation.** Every workflow step is idempotent and shown live; a failed request is fixed and retried, and every change is checked on the devices and from the WAN before it is called done. The Fabric Dashboard shows the whole fabric, the KPIs its design implies and a live check of the network against that design, at any time.

The paper describes the platform, the network design, the policy, the application, the sources of truth, the automation and orchestration in detail, the self-service app, a change end to end, how the lab survives failures, and the full tech stack.

2 Platform and deployment

One Ubuntu host runs every virtual machine and the lab's own services; a shared network management server (NMS) runs the source of truth and the CI system.

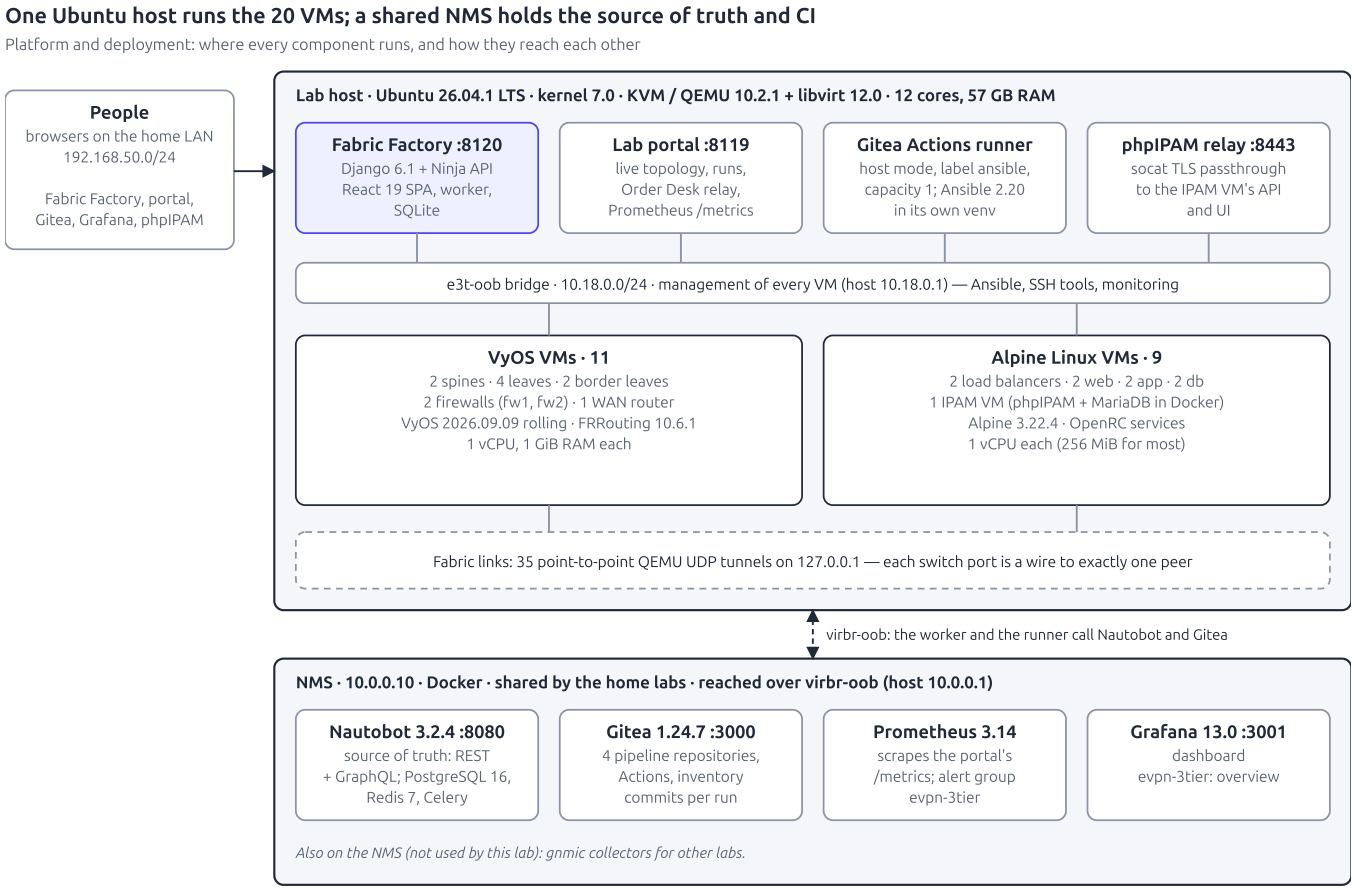


Figure 1. Platform and deployment. The 20 VMs, the lab host's services, and the NMS services the automation depends on.

Each VM is a KVM guest managed by libvirt. The fabric's links are not bridges: every switch port is a QEMU UDP tunnel to exactly one peer port, so the wiring is point-to-point like real cabling and is proved by `lab.sh check-wiring` (MAC addresses and LACP partners). Every VM also has a management interface on the e3t-oob bridge, which the automation, the device tools and the monitoring use; none of that traffic touches the fabric.

Service	Where	Port	Purpose
Fabric Factory	lab host	8120	Self-service app: requests, approval, live progress, device tools, Fabric Dashboard, API
Lab portal	lab host	8119	Live topology, runs, the Order Desk shop, Prometheus metrics
phpIPAM relay	lab host → IPAM VM	8443	TLS passthrough to phpIPAM's UI and API
Gitea Actions runner	lab host	—	Executes the four pipelines in host mode
Nautobot	NMS 10.0.0.10	8080	Network source of truth (REST, GraphQL)
Gitea	NMS 10.0.0.10	3000	Pipeline repositories and Actions

Service	Where	Port	Purpose
Grafana / Prometheus	NMS 10.0.0.10	3001 / 9090	Dashboards and alerting
Lab Hub	lab host	8088	Power, status and links across all home labs

3 The network fabric

A two-tier Clos of VyOS routers: 2 spines, 4 leaves and 2 border leaves, running eBGP unnumbered with BFD in the underlay and EVPN/VXLAN in the overlay.

Every server is dual-homed; every leaf and border has one link to each spine

Fabric topology: AS numbers, loopbacks (VTEP / router ID), management addresses, Ethernet segments (ES)

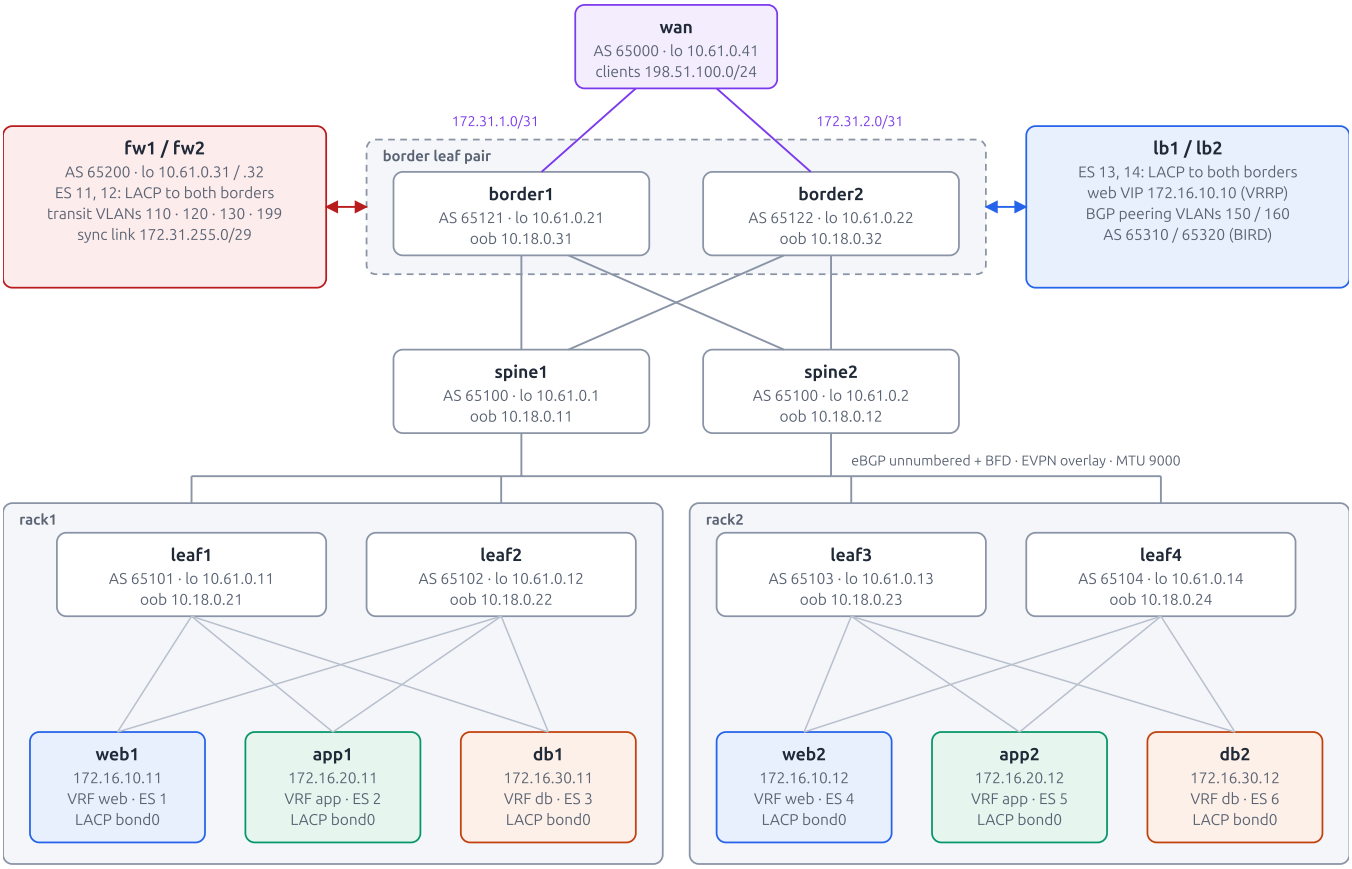


Figure 2. Fabric topology. ESI = Ethernet segment identifier; every server and every border-attached appliance is dual-homed with LACP.

Underlay

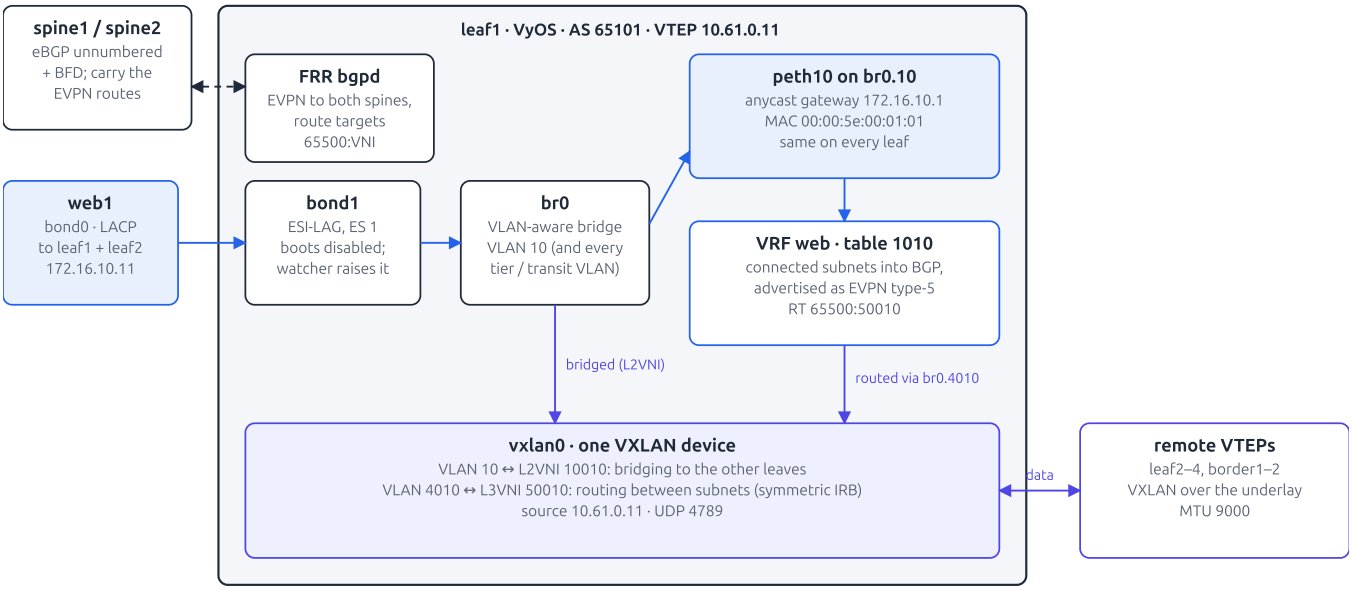
Every leaf and border has one link to each spine. BGP runs over IPv6 link-local addresses (unnumbered), so no point-to-point subnets need planning, and BFD (300 ms × 5) detects a failed link in 1.5 s. The spines share AS 65100; each leaf and border has its own AS. Each switch has a loopback in 10.61.0.0/24 that serves as router ID and VXLAN tunnel endpoint (VTEP). Up to four equal-cost paths are installed, and the underlay MTU is 9000 so VXLAN never fragments.

Overlay

The same BGP sessions carry EVPN routes. Route targets are explicit (65500:VNI) rather than derived from AS numbers, which keeps them identical across switches that each have their own AS. Each tier VLAN maps to a layer-2 VNI (10000 + VLAN); each VRF has a layer-3 VNI for routed traffic.

Inside a leaf: bridging on the L2VNI, routing on the L3VNI, one gateway everywhere

EVPN/VXLAN data and control plane on leaf1 for VLAN 10 (VRF web) — symmetric IRB with an anycast gateway



Dashed: control plane (BGP EVPN to the spines). Route types: 1 and 4 Ethernet segments (multihoming, designated forwarder) · 2 MAC and MAC/IP of every host · 3 VTEP membership per VNI · 5 IP prefixes per VRF.

Figure 3. Inside leaf1 for VLAN 10: the server's bond, the VLAN-aware bridge, the anycast gateway, the VRF and the single VXLAN device.

Symmetric IRB. A leaf routes traffic between subnets itself, in the VRF, and sends routed packets over the VRF's L3VNI; the receiving leaf routes them again into the destination subnet. **Anycast gateway:** every leaf answers for the same gateway IP and virtual MAC (00:00:5e:00:01:01), so a server's gateway is always its own leaf. **Multihoming:** each server's LACP bond spans two leaves as one Ethernet segment; EVPN elects a designated forwarder and withdraws a failed leg within a second. The segment bonds boot disabled and a watcher raises them only once EVPN has converged.

VRF	L3VNI / VLAN	Tier VLANs (L2VNI = 10000 + VLAN)	Firewall transit
web	50010 / 4010	10 · 172.16.10.0/24 (lab design); 41, 43, 44 via Fabric Factory	VLAN 110 · 172.16.110.0/28
app	50020 / 4020	20 · 172.16.20.0/24 (lab design); 40, 42 via Fabric Factory	VLAN 120 · 172.16.120.0/28
db	50030 / 4030	30 · 172.16.30.0/24 (lab design)	VLAN 130 · 172.16.130.0/28
outside	50099 / 4090	none; WAN links 172.31.1.0/31, 172.31.2.0/31	VLAN 199 · 172.16.199.0/28

Rendered configuration on leaf1 for VLAN 10 (an excerpt of what the switches pipeline generates from Nautobot):

```
set interfaces bridge br0 vif 10 address '172.16.10.241/24'
set interfaces bridge br0 vif 10 vrf 'web'
set interfaces pseudo-ethernet peth10 address '172.16.10.1/32'
set interfaces pseudo-ethernet peth10 source-interface 'br0.10'
set interfaces pseudo-ethernet peth10 vrf 'web'
set interfaces vxlan vxlan0 vlan-to-vni 10 vni '10010'
set protocols bgp address-family l2vpn-evpn vni 10010 route-target both '65500:10010'
set vrf name web vni '50010'
set vrf name web table '1010'
set vrf name web protocols bgp address-family ipv4-unicast redistribute connected
set vrf name web protocols bgp address-family l2vpn-evpn advertise ipv4 unicast
set vrf name web protocols bgp address-family l2vpn-evpn route-target both '65500:50010'
```

4 Routing and security policy

The firewalls are the only path between VRFs: routing makes them the only way out of a tier, and the zone firewall decides which connections may start.

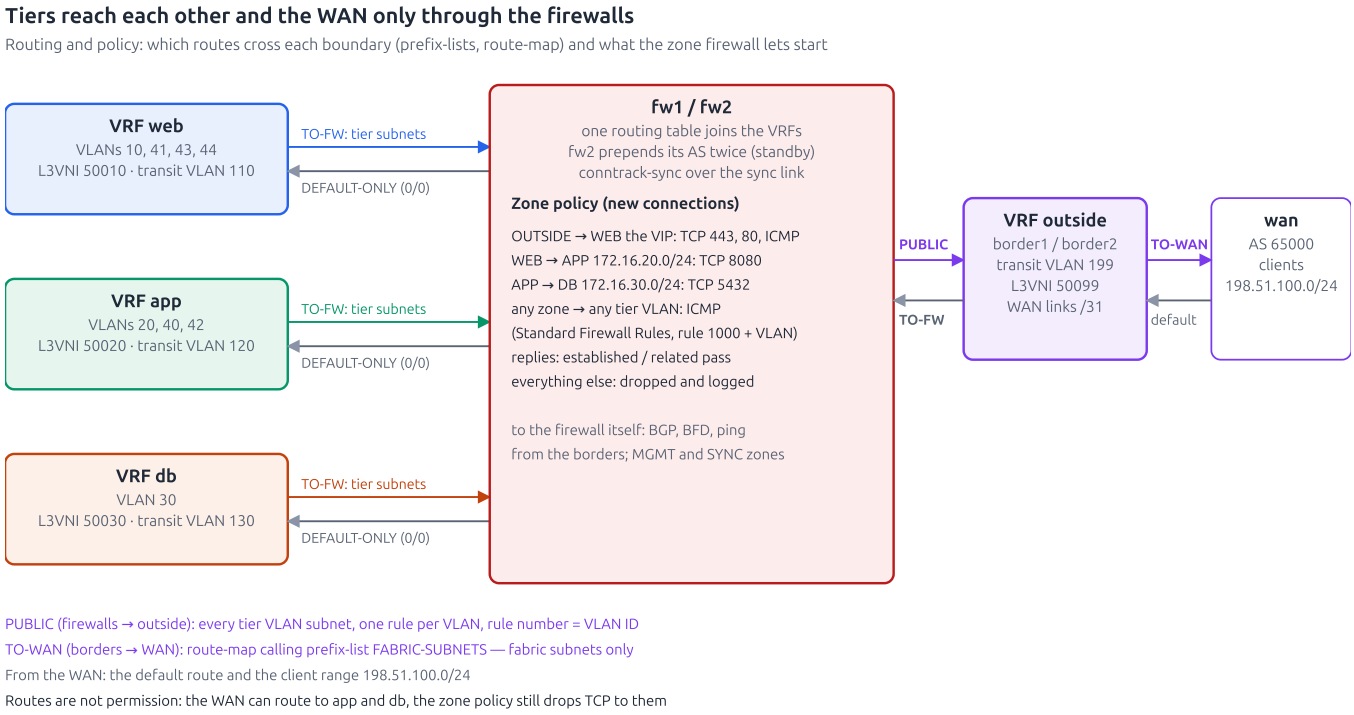


Figure 4. Routing and policy. Arrows show which routes cross each boundary; the zone policy governs traffic, independent of what is routed.

The border pair carries one transit VLAN per VRF down a bond to each firewall. Each firewall runs eBGP to both borders on every transit VLAN and keeps all VRFs in one routing table, so routing between tiers happens inside the firewall, where the zone policy applies. Four policies shape what each side learns:

Policy	Where	Permits	Why
TO-FW (prefix-list)	borders → firewalls	Tier subnets (≤ /24); no transit addresses, no host /32s	EVPN inserts the other border's transit address as a /32; sent on, it re-routed the firewalls' BFD through the wrong border
DEFAULT-ONLY	firewalls → tier VRFs	0.0.0.0/0 only	A tier can leave only through the firewall
PUBLIC	firewalls → outside	Every tier VLAN subnet; rule number = VLAN ID	The WAN learns routes to every tier; traffic is still filtered
TO-WAN → FABRIC-SUBNETS	borders → WAN	Fabric VLAN subnets only	Loopbacks, transit, /31s, /32s and the default route never leave

The zone firewall. VyOS zones WEB, APP, DB and OUTSIDE (one per VRF's transit VLAN) plus SYNC, MGMT and LOCAL. Replies always pass; new connections pass only as listed in Nautobot's `fw_policy`; everything else is dropped and logged. **Standard Firewall Rules** open ICMP to every tier VLAN from every other zone, numbered 1000 + VLAN ID so they never renumber; Add a VLAN creates them and Remove a VLAN deletes them.

Firewall high availability. fw1 is preferred because fw2 prepends its AS twice on everything it advertises. conntrackd replicates connection state to fw2 over a dedicated sync link, tied to a VRRP group that tracks the active firewall.

Powering off fw1 pauses a live connection for 1.8 s (BFD plus BGP) before it continues through fw2.

border1: what the WAN may learn

```
set policy prefix-list FABRIC-SUBNETS rule 10
action 'permit'
set policy prefix-list FABRIC-SUBNETS rule 10
description 'VLAN 10 (VRF web)'
set policy prefix-list FABRIC-SUBNETS rule 10
prefix '172.16.10.0/24'
set policy prefix-list FABRIC-SUBNETS rule 41
prefix '172.16.0.0/24'
set policy route-map TO-WAN rule 10 action
'permit'
set policy route-map TO-WAN rule 10 match ip
address prefix-list 'FABRIC-SUBNETS'
set vrf name outside protocols bgp neighbor
172.31.1.1 address-family ipv4-unicast route-map
export 'TO-WAN'
```

fw1: replies, and a Standard Firewall Rule

```
set firewall ipv4 name OUTSIDE-APP rule 1 action
'accept'
set firewall ipv4 name OUTSIDE-APP rule 1 state
'established'
set firewall ipv4 name OUTSIDE-APP rule 1 state
'related'
set firewall ipv4 name OUTSIDE-APP rule 1020
action 'accept'
set firewall ipv4 name OUTSIDE-APP rule 1020
description 'standard: ICMP to VLAN 20 (app
172.16.20.0/24)'
set firewall ipv4 name OUTSIDE-APP rule 1020
destination address '172.16.20.0/24'
set firewall ipv4 name OUTSIDE-APP rule 1020
protocol 'icmp'
set firewall zone APP from OUTSIDE firewall name
'OUTSIDE-APP'
```

5 The application

Order Desk is a small shop whose every answer names the load balancer, web server, app server and database that served it, so failovers are visible on the page.

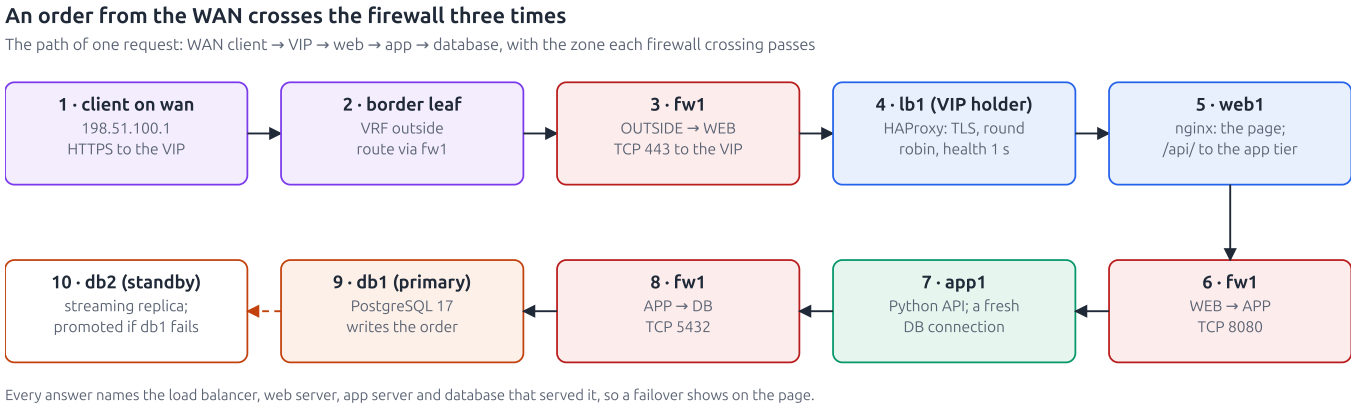


Figure 5. One order from a WAN client. The firewall is crossed three times, each time by a different zone pair.

Tier	Software	How it is made redundant
Load balancers lb1, lb2	HAProxy 3.2 (TLS on the VIP, round robin, 1 s health checks); keepalived 2.3	VRRP moves VIP 172.16.10.10 with a virtual MAC, so no switch keeps a stale entry
Web web1, web2	nginx 1.28: the page, /health, /api/ proxy	HAProxy redispatches; nginx retries the other app server on connection errors
App app1, app2	Python 3.12 API (standard-library HTTP server + psycopg 3), port 8080	A fresh connection per request; the connection string asks for the read-write database
Database db1, db2	PostgreSQL 17, database shop	Streaming replication to a hot standby; promotion and reseeding via lab.sh db

Load balancers that speak BGP

A load balancer can also announce its own VIP prefixes to the fabric. lb1 (AS 65310, 172.16.200.0/28) and lb2 (AS 65320, 172.16.210.0/28) run BIRD 2.17 over dedicated peering VLANs (150, 160) to both border leaves, with BFD. The borders accept only the VIP prefixes a peering was approved for, send the load balancer nothing, and redistribute the VIPs into the web VRF as EVPN type-5 routes. Both peerings were requested and built through Fabric Factory.

6 Sources of truth and what is in Nautobot

Two systems hold the truth, each for one kind of fact; everything on the devices is derived from them.

System	Holds	Written by	Read by
phpIPAM 1.8.3	Addressing: the evpn-3tier section, subnets under 172.16.0.0/16, gateway and host reservations, VLAN numbers, VRFs	Fabric Factory (reserves first, releases last); the initial loader	Fabric Factory's allocator (free /24s, used VLANs)
Nautobot 3.2.4	The plan: devices, interfaces, cabling, roles, racks; VLAN and prefix objects; the evpn-3tier config context (VRFs, VLANs, transit, segments, WAN, firewall policy, load balancer peerings)	Fabric Factory; the initial seed from lab.conf	Every pipeline (GraphQL); Fabric Factory's lab model and form validation
Gitea 1.24.7	Four repositories: templates, playbooks, the inventory each run rendered, and the run history	People (templates); every pipeline run (inventory commit)	The runner

The config context is the heart of the plan. Pipelines render every device from it; Fabric Factory edits it. An excerpt:


```
{
  "evpn": {
    "rt_as": 65500, "anycast_mac": "00:00:5e:00:01:01",
    "vrfs":    [{"name": "web", "l3vni": 50010, "l3vlan": 4010, "table": 1010}, ...],
    "vlans":   [{"vlan": 10, "vrf": "web", "vni": 10010, "gateway": "172.16.10.1/24", "on": ["leaves",
"borders"]}, ...],
    "transit": [{"vlan": 110, "vrf": "web", "vni": 10110, "subnet": "172.16.110.0/28"}, ...]
  },
  "fw_policy": [
    {"from": "outside", "to": "web", "dest": "172.16.10.10", "proto": "tcp", "port": "443"},
    {"from": "outside", "to": "app", "dest": "172.16.20.0/24", "proto": "icmp", "port": null,
     "standard": true, "vlan": 20}, ...
  ],
  "lb_peerings": [{"name": "lb1", "asn": 65310, "vlan": 150, "vni": 10150, "subnet": "172.16.150.0/29",
                   "vips": ["172.16.200.0/28"], "vrf": "web", "switches": ["border1", "border2"], "bfd":
true}],
  "wan": {"clients": "198.51.100.0/24", ...}, "web_vip": "172.16.10.10", "fabric_mtu": 9000, "max_paths":
4
}
```


Why both. phpIPAM is the authority on which addresses are free, so allocation and reservation happen there first; Nautobot is the authority on what the network should be, so the pipelines read only Nautobot. Fabric Factory writes phpIPAM before Nautobot when adding, and releases phpIPAM after the devices are clean when removing, so an address is never handed out while still in use.

Where the lab lives in Nautobot

The NMS's Nautobot is shared by several labs, so everything of this one sits under five names and never mixes with the others: the location, the namespace, the VLAN group and the tenant group are all called **evpn-3tier**, and so is the config context; device names carry the prefix **e3t-** (Nautobot names are global). `tools/nautobot_seed.py` creates all of it from lab.conf (idempotent; it also removes stale objects it owns), and from then on the workflows change it.

Object	Where in Nautobot	Count	Created and changed by
Location	Organization → Locations → **evpn-3tier** (type Site)	1	Seed
Racks	Organization → Racks, location evpn-3tier: spine, rack1, rack2, border, edge, wan	6	Seed
Devices	Devices → Devices, location evpn-3tier. Name e3t-<node>, role e3t-<role>, device type VyOS or Alpine host, platform vyos or alpine, rack, primary IPv4 = the OOB address (Ansible's ansible_host); local config context: index, ASN, console port, Ethernet segment	19	Seed
Interfaces	On each device: eth0 (management only), eth1–7 with the lab's MACs, descriptions and MTU 9000 on fabric links; dum0 with the loopback /32; bond0 and bond0.<VLAN> on the servers	117	Seed
Cables	Every link of lab.conf, labelled with its kind: fabric, access, wan or sync	35	Seed
IP addresses	IPAM → IP Addresses, namespace evpn-3tier: OOB addresses, loopbacks, the servers' tier addresses	38	Seed
VRFs and route targets	IPAM → VRFs, namespace evpn-3tier: web, app, db, outside, each with tenant e3t-<VRF> and route target 65500:<L3VNI> (import and export)	4 + 4	Seed
Tenants	Tenancy → Tenants, group evpn-3tier: e3t-web, e3t-app, e3t-db, e3t-outside	4	Seed
VLANs	IPAM → VLANs, group evpn-3tier: tier VLANs, firewall transit, load balancer peering, the L3VNI VLANs	21	Seed; Add a VLAN and LB peering create, Remove a VLAN deletes
Prefixes	IPAM → Prefixes, namespace evpn-3tier, each with its VRF, VLAN, tenant and location	25	Seed; Add a VLAN and LB peering create, Remove a VLAN deletes
Config context	Extensibility → Config Contexts → **evpn-3tier**, scoped to the location: `evpn` (VRFs, VLANs with VNI, gateway and span, transit), `fw_policy` (38 rules), `lb_peerings` (2), `wan`, `es`, `oob`, `web_vip`, MTU and ECMP settings	1	Seed; every workflow edits it

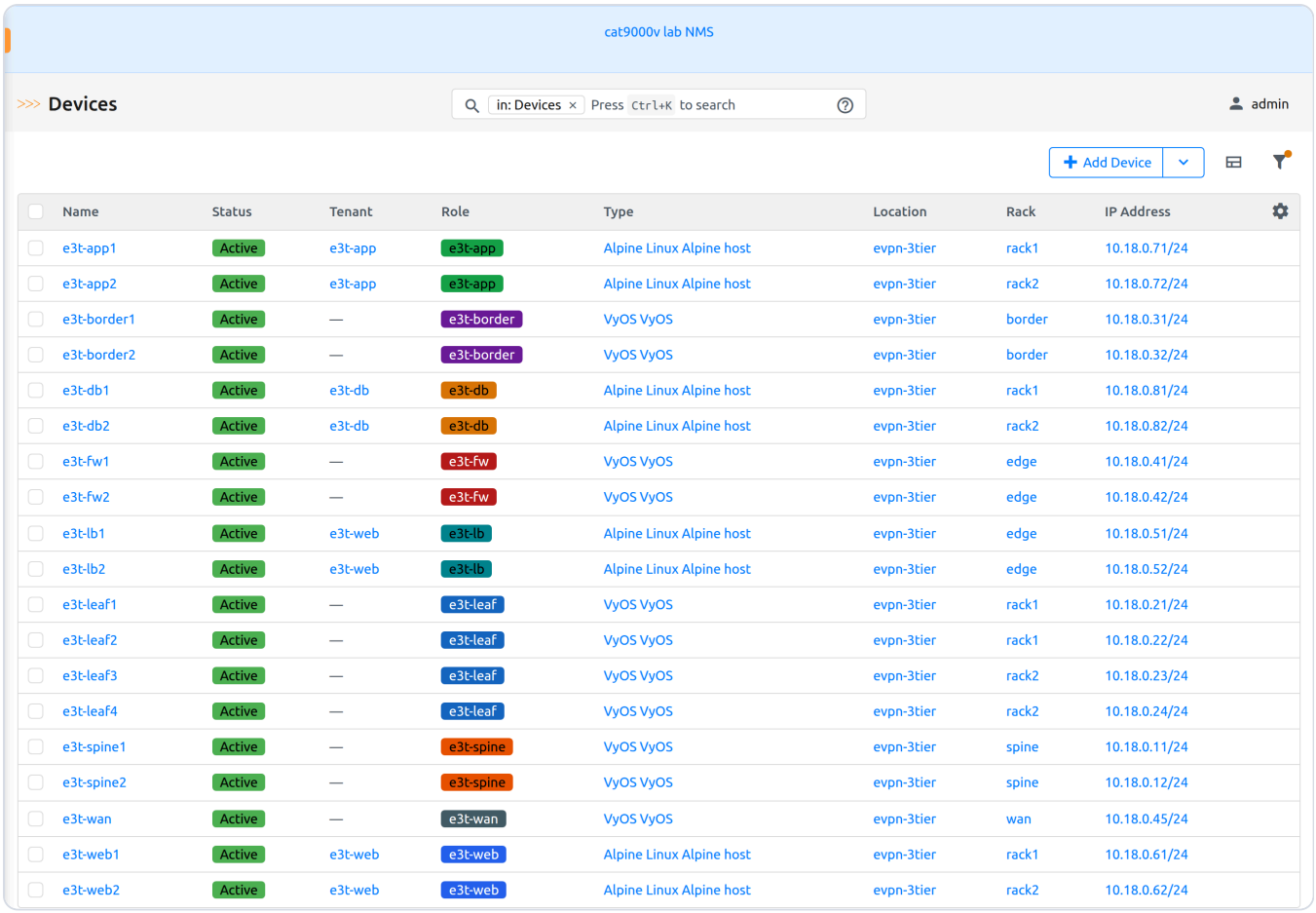


Figure 6. The 19 devices in location evpn-3tier: role, device type, rack and the management address the pipelines connect to.

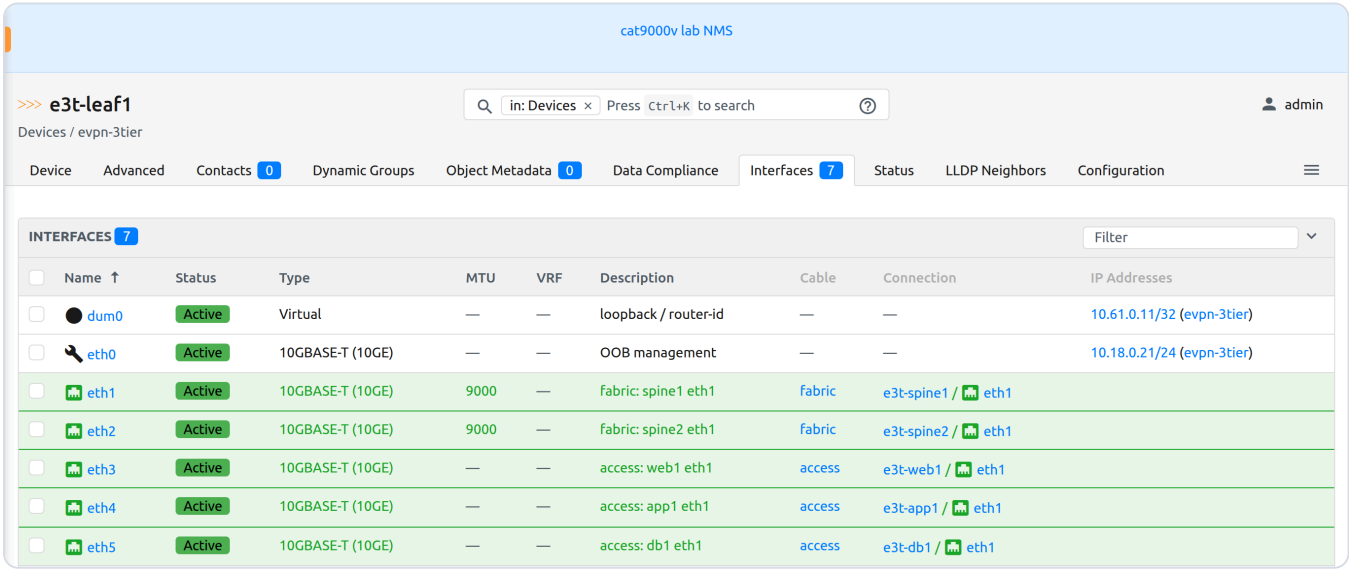
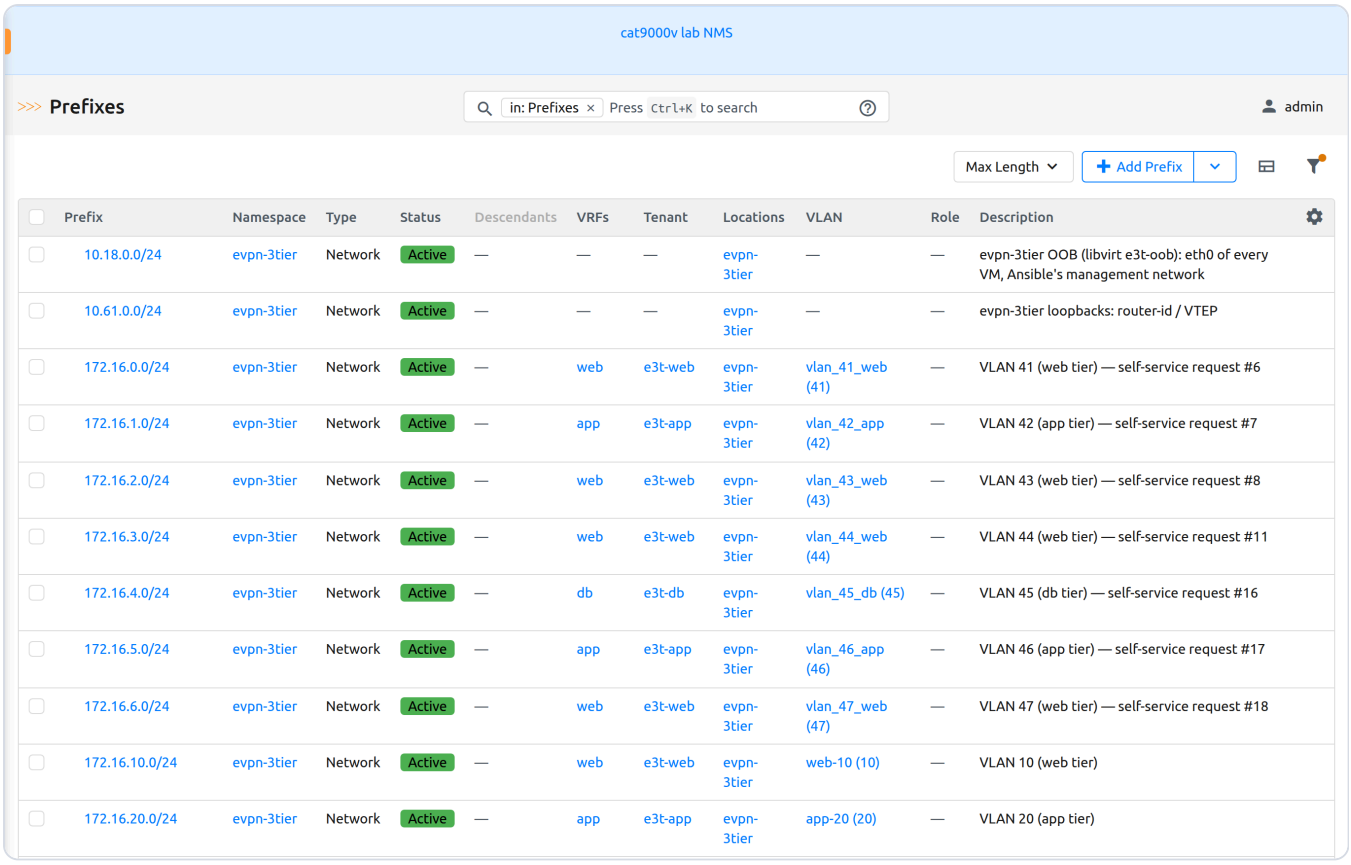


Figure 7. e3t-leaf1's interfaces and cables: the uplinks to both spines (fabric, MTU 9000) and the three server legs (access). The pipelines, the topology view and the switch-pair logic are all built from these cables.



Prefix	Namespace	Type	Status	Descendants	VRFs	Tenant	Locations	VLAN	Role	Description
10.18.0.0/24	evpn-3tier	Network	Active	—	—	—	evpn-3tier	—	—	evpn-3tier OOB (libvirt e3t-oob): eth0 of every VM, Ansible's management network
10.61.0.0/24	evpn-3tier	Network	Active	—	—	—	evpn-3tier	—	—	evpn-3tier loopbacks: router-id / VTEP
172.16.0.0/24	evpn-3tier	Network	Active	—	web	e3t-web	evpn-3tier	vlan_41_web (41)	—	VLAN 41 (web tier) — self-service request #6
172.16.1.0/24	evpn-3tier	Network	Active	—	app	e3t-app	evpn-3tier	vlan_42_app (42)	—	VLAN 42 (app tier) — self-service request #7
172.16.2.0/24	evpn-3tier	Network	Active	—	web	e3t-web	evpn-3tier	vlan_43_web (43)	—	VLAN 43 (web tier) — self-service request #8
172.16.3.0/24	evpn-3tier	Network	Active	—	web	e3t-web	evpn-3tier	vlan_44_web (44)	—	VLAN 44 (web tier) — self-service request #11
172.16.4.0/24	evpn-3tier	Network	Active	—	db	e3t-db	evpn-3tier	vlan_45_db (45)	—	VLAN 45 (db tier) — self-service request #16
172.16.5.0/24	evpn-3tier	Network	Active	—	app	e3t-app	evpn-3tier	vlan_46_app (46)	—	VLAN 46 (app tier) — self-service request #17
172.16.6.0/24	evpn-3tier	Network	Active	—	web	e3t-web	evpn-3tier	vlan_47_web (47)	—	VLAN 47 (web tier) — self-service request #18
172.16.10.0/24	evpn-3tier	Network	Active	—	web	e3t-web	evpn-3tier	web-10 (10)	—	VLAN 10 (web tier)
172.16.20.0/24	evpn-3tier	Network	Active	—	app	e3t-app	evpn-3tier	app-20 (20)	—	VLAN 20 (app tier)

Figure 8. Prefixes in namespace evpn-3tier. Each subnet a request added names the request in its description and is tied to its VRF, tenant, location and VLAN.

How Fabric Factory writes to Nautobot

The worker uses Nautobot's REST API with a token read from the NMS at run time. Every write is preceded by a read of the same object (a GET by name, VLAN ID or prefix), so a retry finds what an earlier attempt created and skips it; the config context is always read, changed and written back whole.

Workflow	REST calls, in order	What they do
Add a VLAN	GET + PATCH /api/extras/config-contexts/{id}/	Append {vlan, vrf, vni, gateway, on} to `evpn.vlans`, after checking that no VNI or subnet in the plan clashes
	POST /api/ipam/vlans/	VLAN object: VID, name vlan_<ID>_<VRF>, group evpn-3tier, tenant, a description naming the request
	POST /api/ipam/prefixes/, PATCH …/prefixes/{id}/, POST /api/ipam/vrf-prefix-assignments/	The subnet in namespace evpn-3tier with its VLAN and tenant; then its location; then its VRF
	GET + PATCH /api/extras/config-contexts/{id}/	The Standard Firewall Rules: three ICMP entries in `fw_policy`, tagged standard and with the VLAN
Remove a VLAN	GET + PATCH config context (twice)	The VLAN out of `evpn.vlans`; its tagged entries out of `fw_policy`
	DELETE /api/ipam/prefixes/{id}/, DELETE /api/ipam/vlans/{id}/	The prefix and the VLAN object
LB peering	GET + PATCH config context	Append the peering to `lb_peerings` (ASN, switches, VLAN, VNI, subnet, VIPs, BFD)

Workflow	REST calls, in order	What they do
	POST vlans; POST prefixes + PATCH location + POST vrf-prefix-assignments	VLAN peering-<LB>; the peering /29 and each VIP prefix in the VRF

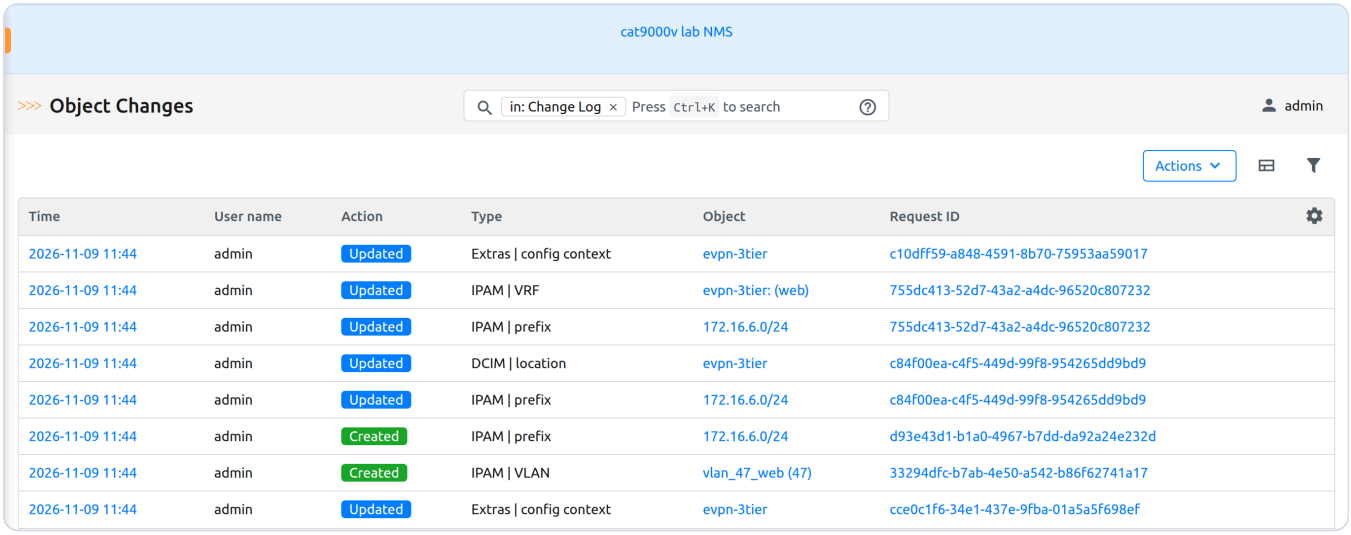


Figure 9. Nautobot's change log holds every write. Request #18 (Add VLAN 47), newest first: the plan entry (bottom), the VLAN and the prefix created, the prefix tied to its location and VRF, and the Standard Firewall Rules (top).

How it is read back: GraphQL

Nothing walks the REST API object by object to rebuild the plan: each reader sends one GraphQL query to `/api/graphql/` and gets the devices, their cabling and the config context in a single answer.

Reader	When	Asks for	Turns it into
Pipeline inventory (`nautobot_inventory.py` in each Gitea repository)	Every pipeline run	The config context; every device of the location with role, rack, platform, primary IPv4, local context, and every interface's description, MAC, MTU, VRF, addresses, cable label and cable peer	hosts.yml, group_vars (the whole lab model) and host_vars, committed to the repository; the templates render from it
Fabric Factory's lab model (`labmodel.py`)	Form validation, allocation, removal checks (cached 30 s)	The config context; devices with role, rack and local context; the servers' bond interfaces with VRF and cable peers; the VLANs of the group	VLAN IDs, VNIs, subnets and ASNs in use; switch pairs; each load balancer's pair and VRF; which VLANs may be removed
Fabric Dashboard (`fabric.py`)	When the page opens (cached 30 s)	The above plus device types, platforms, statuses, interface types, VRFs and prefixes	Counts, design KPIs, the topology
The executors' guards	Before every write	REST GETs by name, VID or prefix	Idempotence: what already exists is skipped

The pipelines' query, in full:

```
query {
  config_contexts(name: ["evpn-3tier"]) { data }
  devices(location: ["evpn-3tier"]) {
    name local_config_context_data
    role { name } rack { name } platform { name } primary_ip4 { address }
    interfaces {
      name description mac_address mtu
      vrf { name }
      ip_addresses { address }
      cable { label }
      cable_termination { cable_end }
      cable_peer_interface { name device { name } }
    }
  }
}
```

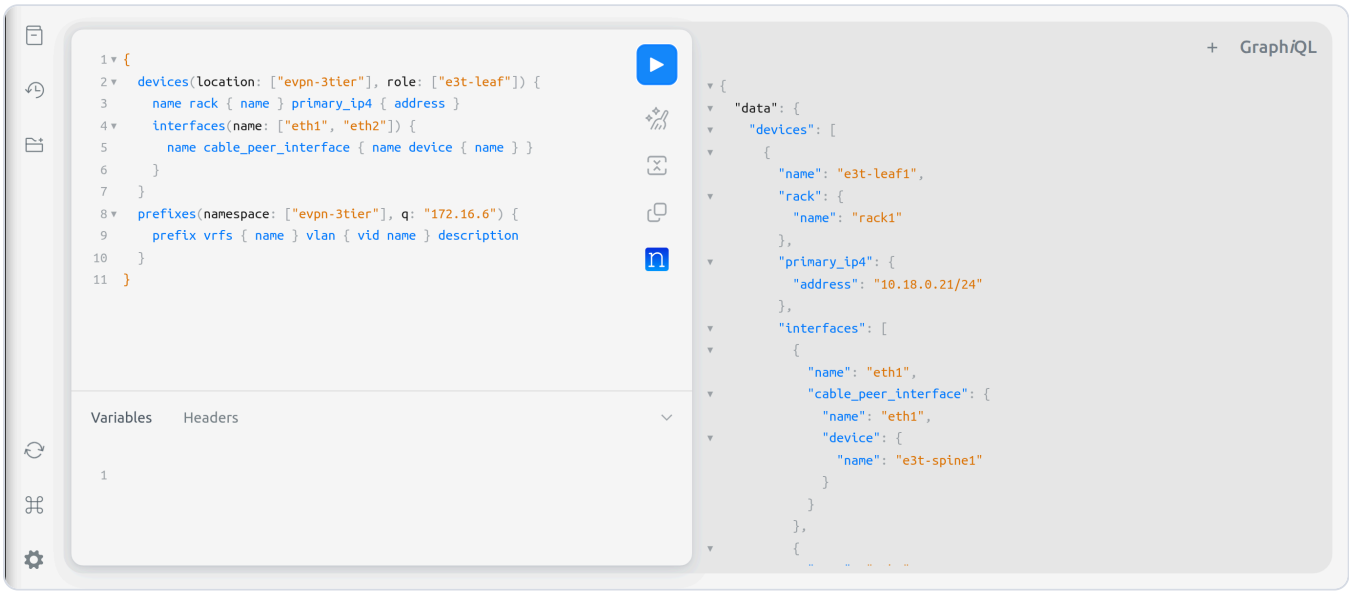


Figure 10. The same kind of query in Nautobot's GraphiQL: the leaves with their racks, management addresses and uplink cables, and the prefix of VLAN 47 with its VRF and VLAN.

7 Automation and orchestration

Fabric Factory orchestrates the systems in order; the pipelines are the only thing that configures a device; Ansible renders and applies, and deletes what intent no longer contains.

Fabric Factory orchestrates; the pipelines configure; nothing writes to a device by hand

Automation and orchestration: components, the interfaces between them, and what each call carries

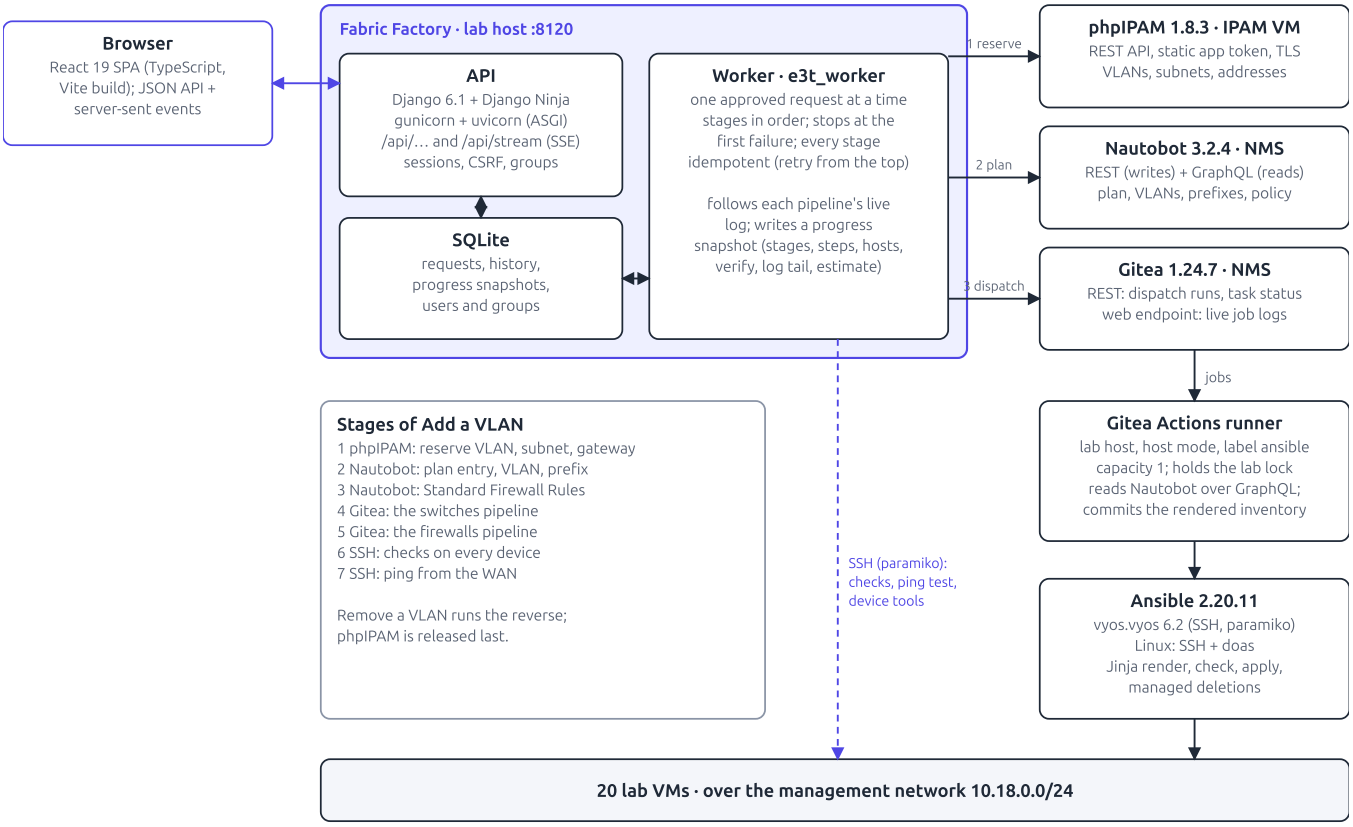


Figure 11. Automation and orchestration. Solid arrows: requests and writes; dashed: reads and observation.

Interfaces between the components

From → to	Protocol and authentication	Carries
Browser → Fabric Factory	Plain HTTP in the lab; Django session cookie, CSRF token on every change	JSON API calls; server-sent events with request changes (1 s)
Worker → phpIPAM	REST over TLS, static application token (ssl_code app)	Free subnets and VLANs; reserve and release VLANs, subnets, addresses
Worker → Nautobot	REST (writes) and GraphQL (reads), API token	Config context edits; VLAN and prefix objects; the lab model
Worker → Gitea	REST with basic auth: dispatch a workflow, poll tasks; the web UI's job endpoint with log cursors	Run start, status, and the live log (only new lines each poll)
Pipeline → Nautobot	GraphQL, token from a Gitea Actions secret	Devices, interfaces, cables, config context → Ansible inventory
Pipeline → Gitea	git over HTTP, the job's own token	The rendered inventory, committed with [skip ci]

From → to	Protocol and authentication	Carries
Ansible → devices	SSH on the management network; VyOS via vyos.vyos (paramiko), Linux via SSH + doas	Configuration lines, files, services
Worker / tools → devices	SSH (paramiko) on the management network	Post-change checks, the ping test, device configurations and state

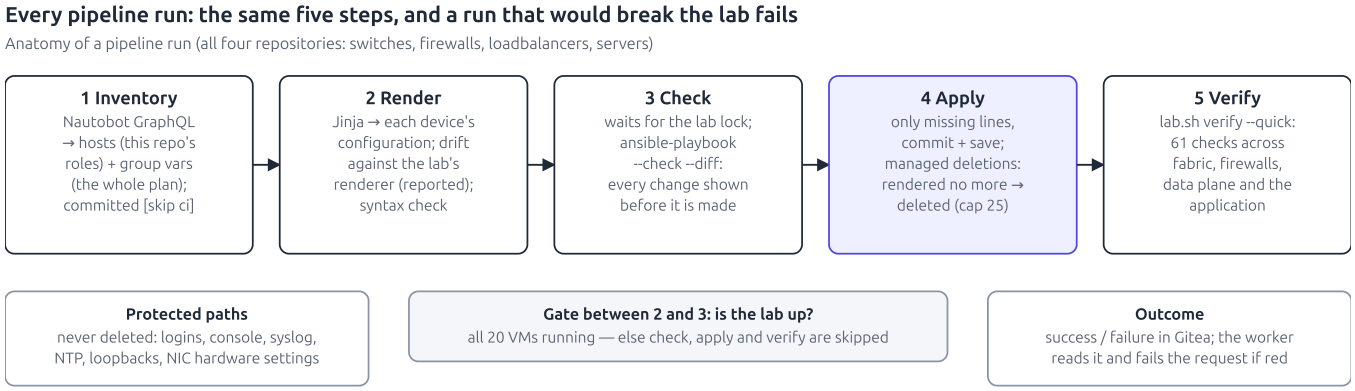


Figure 12. Anatomy of a pipeline run. The same five steps in all four repositories.

How the pipelines work

- **One repository per device group.** evpn-3tier-switches (spines, leaves, borders, wan), evpn-3tier-firewalls, evpn-3tier-loadbalancers and evpn-3tier-servers. Each deploys only its own devices, so a change to firewall templates cannot touch a switch.
- **Inventory from Nautobot, every run.** A Python script queries GraphQL and writes hosts (only this repository's roles) and group variables (the whole plan, so templates can see neighbours). The result is committed back to the repository, so each run's exact input is versioned and diffable.
- **Render, then check, then apply.** Jinja templates render each device's configuration. Check mode (`--check --diff`) records exactly what will change. On VyOS only lines missing from the running configuration are committed; the run is idempotent and a run with nothing to do reports changed=0.
- **Managed deletions.** After the push, a filter compares the running configuration with the rendered one and deletes every line the templates no longer produce, except under protected paths (logins, console, syslog, NTP, loopbacks, NIC hardware settings). A run that would delete more than 25 lines on one device stops for a human. This is what makes removal by intent safe.
- **Verify.** The lab's own verify suite (61 checks in quick mode) runs after every apply; a red verify fails the run.
- **Concurrency.** The runner has capacity 1, and every pipeline and every lab test run takes the same file lock, so two changes, or a change and a test, never overlap.
- **Secrets.** The Nautobot token is a Gitea Actions secret; the worker reads the Gitea and Nautobot credentials from the NMS at run time and phpIPAM's from a mode-600 file on the host. None is stored in a repository.

How Fabric Factory orchestrates

- **Ordered, idempotent stages.** Each workflow is a fixed list of stages. Each stage first checks what already exists, so a retry after a failure skips completed work: a reservation tagged with the request number is recognised as its own.
- **Guards before writes.** Every check runs before the first write to a system, so a refusal changes nothing (for example, a VLAN taken in phpIPAM after the request was submitted).

- **Following the pipelines live.** Gitea 1.24 offers no API for a running job's log, so the worker signs in to Gitea's web UI and polls the job endpoint with per-step log cursors every 3 s. It parses Ansible's PLAY and TASK lines, per-host results and verify's PASS and FAIL lines into a progress snapshot, which the browser receives through server-sent events.
- **Proof, not assumption.** Each workflow ends with checks on the devices themselves and, for VLANs, a ping test from the WAN; the request is done only when they pass.

The progress snapshot stored on a running request (abridged):

```
{
  "stages":    [{"key": "phpipam", "label": "phpIPAM reservation", "status": "done", "started": "…",
"finished": "…"}, …],
  "pipelines": {"evpn-3tier-switches#23": {
      "status": "running", "url": "…/actions/runs/23", "eta_s": 175,
      "steps":  [{"name": "4 apply", "status": "running", "duration": "37s",
                  "hosts": {"leaf1": {"ok": 11, "changed": 1, "failed": 0, "skipped": 0}, …}}, …],
      "play": "VyOS configuration from the templates", "task": "Push templates/vyos/config.j2 …",
      "verify": {"passed": 41, "failed": 0, "total": null, "failures": []},
      "tail": ["4 apply| changed: [leaf1]", …]}},
  "attempt": 1, "seq": 214, "current": "switches"
}
```

8 Fabric Factory, the self-service app

Users request changes in forms checked against the live lab; operators approve; a worker carries them out while everyone watches. Read-only tools show any device's configuration and state; the Fabric Dashboard shows the whole fabric at a glance; the ASN and VLAN databases list every AS and VLAN, downloadable as Excel. Port Provisioning enables, disables and sets up leaf ports.

Most fields fill themselves, by design

> **When a user requests a VLAN, the system fills in most of the form automatically, without being asked.** The requester picks only the VRF (and, optionally, the span) and writes a reason. One button, *Select the next unused VLAN ID*, makes the app choose the next available VLAN ID and the next free subnet, and every value that follows from them. This is deliberate: hand-typed VLAN IDs, VNIs and subnets are where network changes most often go wrong, through a typo, a duplicate ID, or an overlap with a subnet already in use. Values the system allocates come from the sources of truth, cannot be edited in the form, and are checked again at submission and at execution.

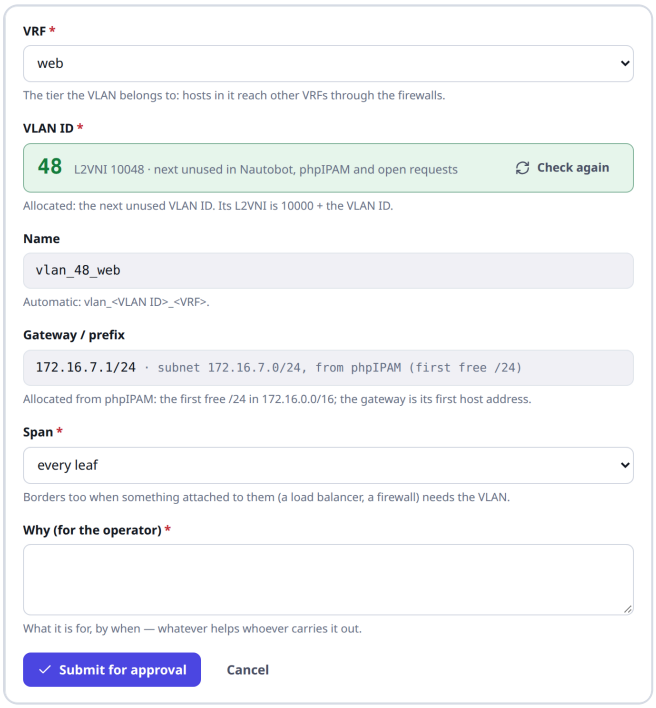


Figure 13. The Add a VLAN form after one click. Only the VRF, the span and the reason were chosen by the user.

Field	Filled in by	How
VRF	**Requester**	web, app or db
Span	**Requester**	Every leaf (default), or leaves and borders
Reason	**Requester**	Free text for the operator
VLAN ID	System	Lowest unused ID, checked against Nautobot, phpIPAM and every open request
L2VNI	System	10000 + VLAN ID
Name	System	vlan_<ID>_<VRF>, e.g. vlan_48_web
Subnet	System	phpIPAM's first free /24 in 172.16.0.0/16 that overlaps nothing
Gateway	System	Always the subnet's first host address

Re-checked at submission and again when the worker carries the request out, so a value taken in the meantime is caught before anything is written.

Workflow	The user chooses	The app works out	Stages
Add a VLAN to the fabric	VRF, span (leaves or leaves + borders), reason	Next unused VLAN ID (and VNI), a free /24 from phpIPAM, gateway = first host, name vlan_ID_VRF	phpIPAM → Nautobot → Standard Firewall Rules → switches → firewalls → checks → ping from the WAN

Workflow	The user chooses	The app works out	Stages
Remove a VLAN from the fabric	The VLAN, a confirmation, reason	Whether it may go: not the lab's design, no hosts, not its VRF's last VLAN, only the gateway reserved in phpIPAM	pre-flight → Nautobot → switches → firewalls → checks → phpIPAM release
Set up a load balancer to peer with the fabric	The load balancer, ASN, peering VLAN and subnet, VIP prefixes, BFD	The switch pair and VRF from the load balancer's cabling in Nautobot	Nautobot → switches → load balancers → phpIPAM → BGP, route and HTTP checks

A request is carried out only after approval, and a failed one can be retried

Request lifecycle in Fabric Factory

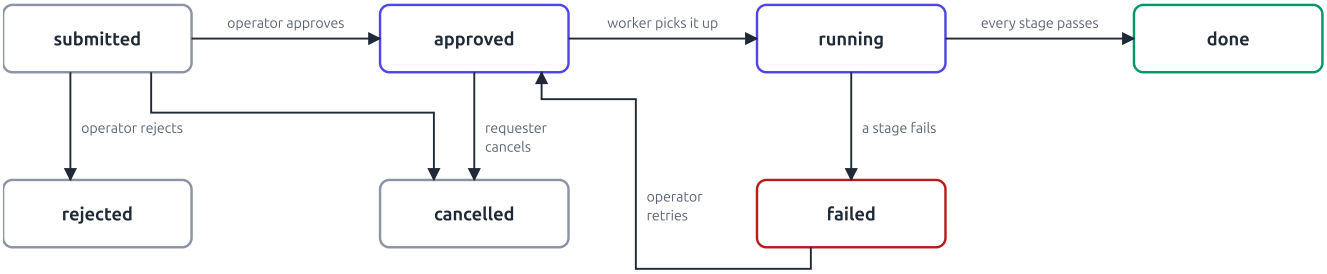


Figure 14. Request lifecycle. Only approved requests are carried out; failed requests are retried after the cause is fixed.

The Fabric Dashboard

The third tool on the New request page, next to the device configurations and device state, and like them read-only, so no approval is needed. It answers two questions at once: *what is this fabric made of*, and *does the running network still match its design*.

- **The design, from Nautobot.** One GraphQL query returns every device with its role, model, platform, rack and cabling, plus the plan's VRFs, VLANs, transit networks and load balancer peerings. The page shows the counts (8 switches: 2 spines, 4 leaves, 2 border leaves; 19 devices; 4 VRFs; 25 subnets; 11 tier VLANs; 35 cabled links; 12 ASNs; 38 firewall rules), every device by role and by model, each VRF with its L3 VNI, VLANs, subnets and servers, the subnets by kind, and a topology drawn from the cables (hovering over a device traces its links).
- **Design KPIs, computed from the cabling.** Listed in the table below; each is green, amber or red against a simple rule.
- **Live health vs. design.** Every device is asked over SSH, read-only and from a fixed command list (BGP and BFD summaries, EVPN VNIs, the software version, uptime). The results are compared with the design: one spine EVPN session per spine link, every BGP and BFD session up, the same VNIs on every leaf, one network software version.
- **Change KPIs** from Fabric Factory's own requests: changes completed, the success rate, the median time from approval to a verified change, and open requests.

Both halves are cached for 30 seconds; the design is reloaded when a request finishes, so a new VLAN appears as soon as it is verified.

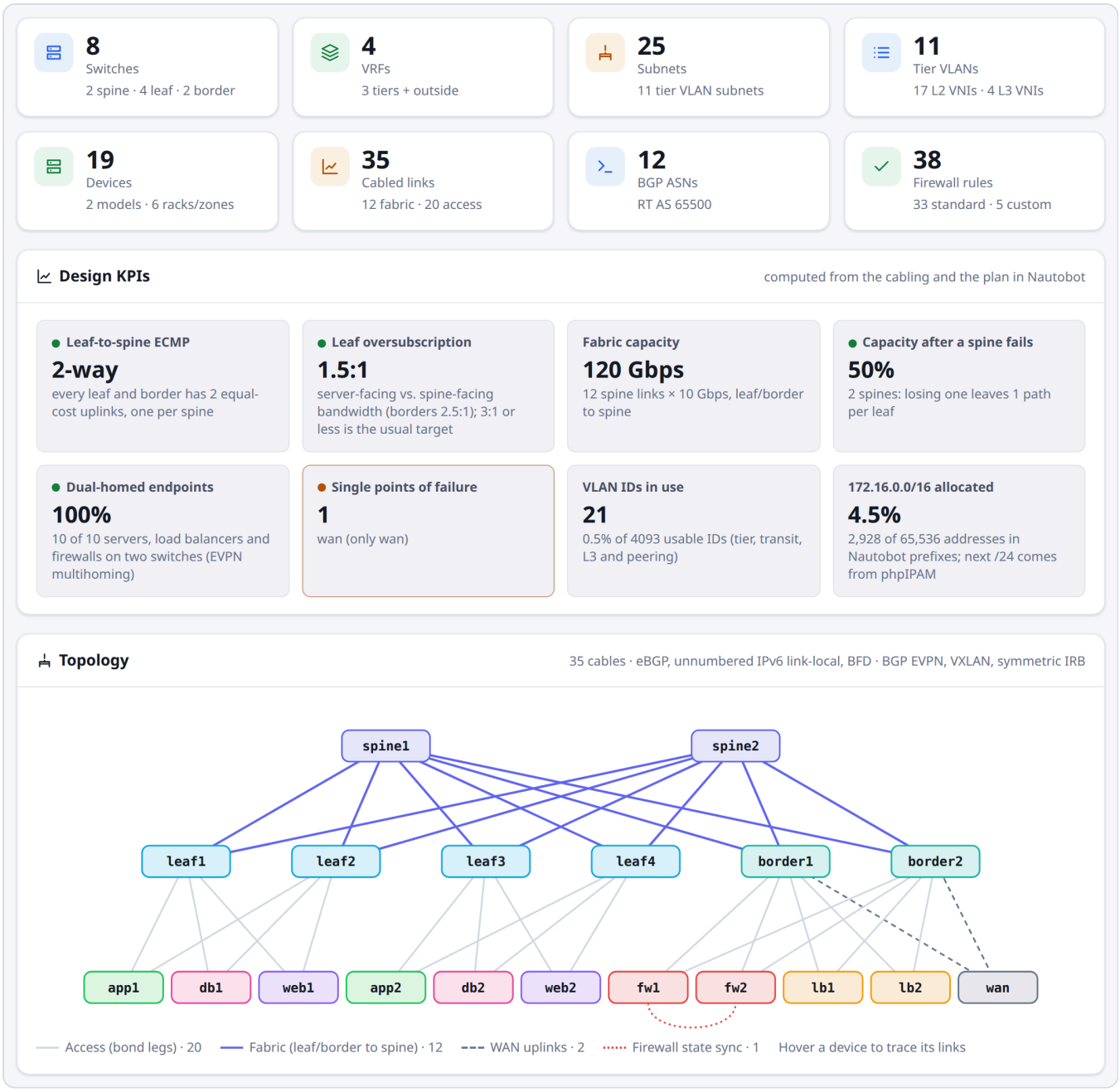


Figure 15. The Fabric Dashboard: the fabric in numbers, the design KPIs and the topology, all from Nautobot.

Design KPI	Today	How it is worked out
Leaf-to-spine ECMP	2-way	Uplinks to spines per leaf and border (the minimum)
Leaf oversubscription	1.5:1	Server-facing over spine-facing bandwidth; borders 2.5:1; target 3:1 or less
Fabric capacity	120 Gbps	12 spine links × 10 Gbps
Capacity after a spine fails	50%	(spines − 1) / spines
Dual-homed endpoints	100%	Servers, load balancers and firewalls cabled to two switches
Single points of failure	1	Roles with one device: wan, which stands in for the internet
VLAN IDs in use	21	Tier, transit, L3 and peering VLANs, of 4,093
172.16.0.0/16 allocated	4.5%	Addresses in Nautobot prefixes

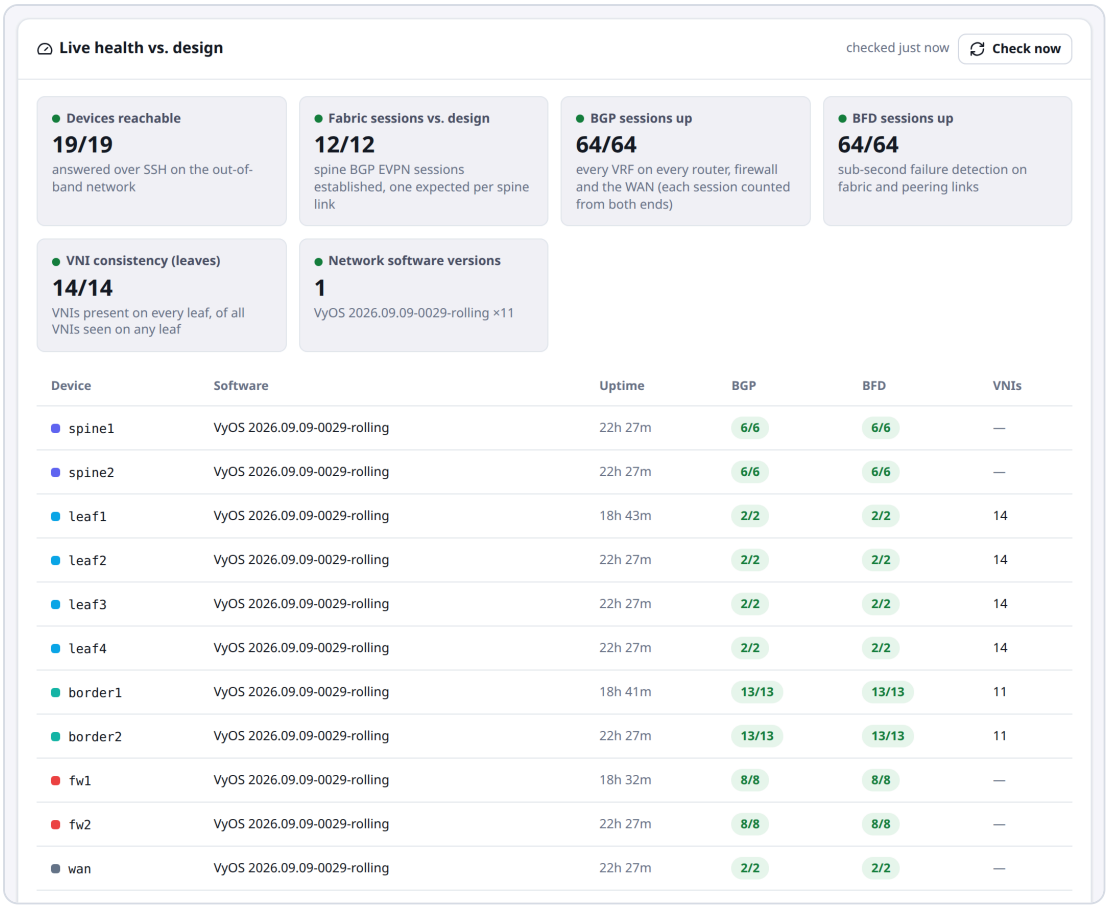


Figure 16. Live health vs. design, checked over read-only SSH: every session up and every leaf carrying the same VNIs.

The ASN and VLAN databases

Two more read-only tools answer the questions asked most often about a fabric: *which AS is that*, and *what is on VLAN 42*. Both read Nautobot's model — the same one the pipelines render from — so they are never out of date, and both download as an Excel workbook (built on the server with openpyxl: a bold header row that stays in place, a filter on every column, numbers stored as numbers, and an About sheet saying where and when the data came from).

Tool	Shows	Excel sheets
ASN Database	All 12 ASNs: the spines (65100, shared), each leaf and border, the firewall pair (65200, shared), the WAN (65000), each load balancer (65310, 65320) and the route-target AS (65500, not a BGP speaker). For each: the devices, roles, racks, router IDs, the ASNs it peers with, its sessions and VRFs. Then all 34 BGP sessions of the design: underlay and EVPN on every spine link, firewall transit in every VRF, the WAN, the load balancer peerings — both ends' addresses, the VRF, BFD.	ASNs, BGP sessions, About
VLAN Database	All 20 VLANs of the evpn-3tier group: tier VLANs, firewall transit, load balancer peering and the L3VNI VLANs. For each: ID, name, type, VRF, VNI, subnet, gateway, usable addresses, where it lives (every leaf, the borders, the firewalls), its hosts or members, and its origin — the lab's design, or a link to the self-service request that made it.	VLANs, About

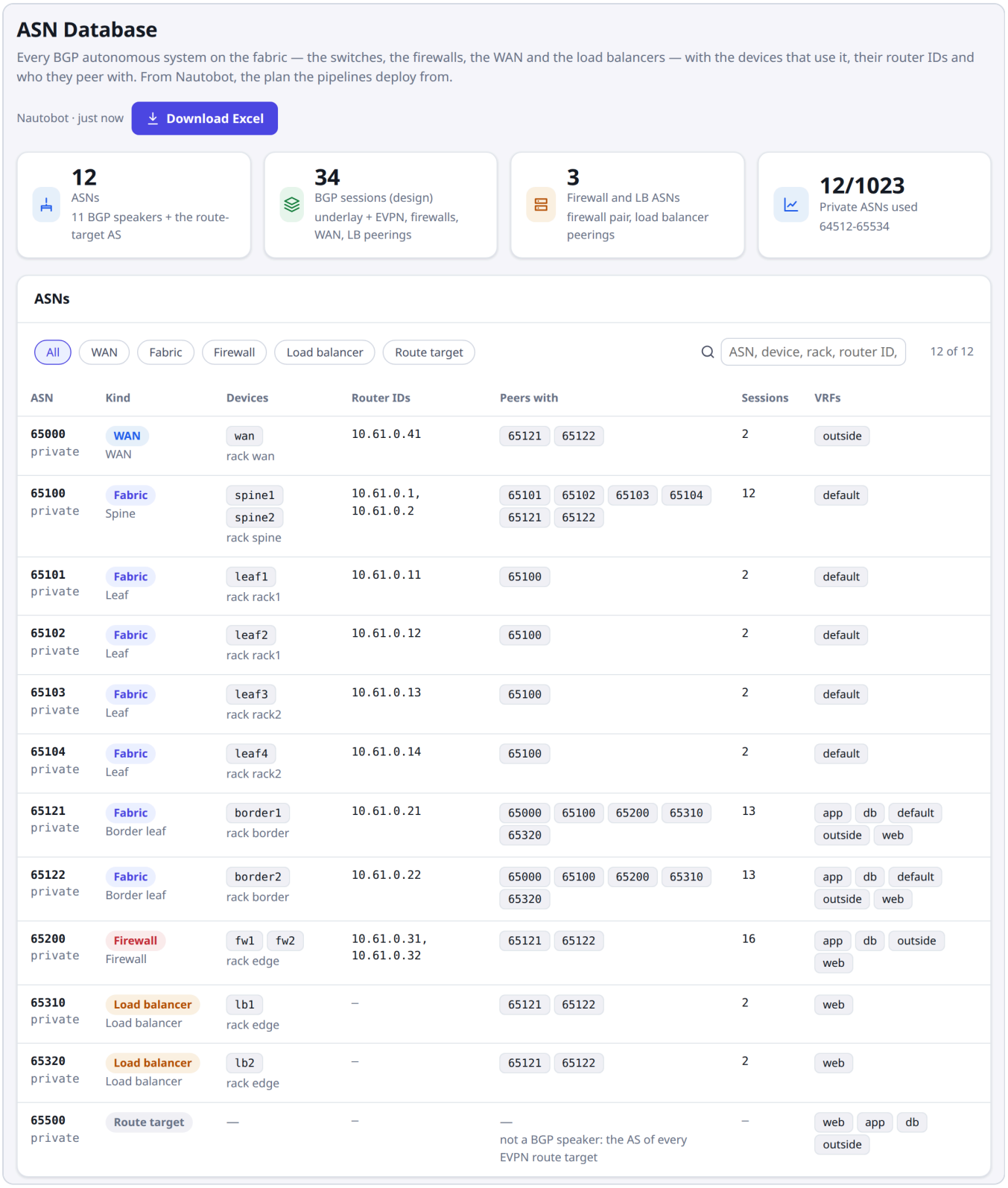
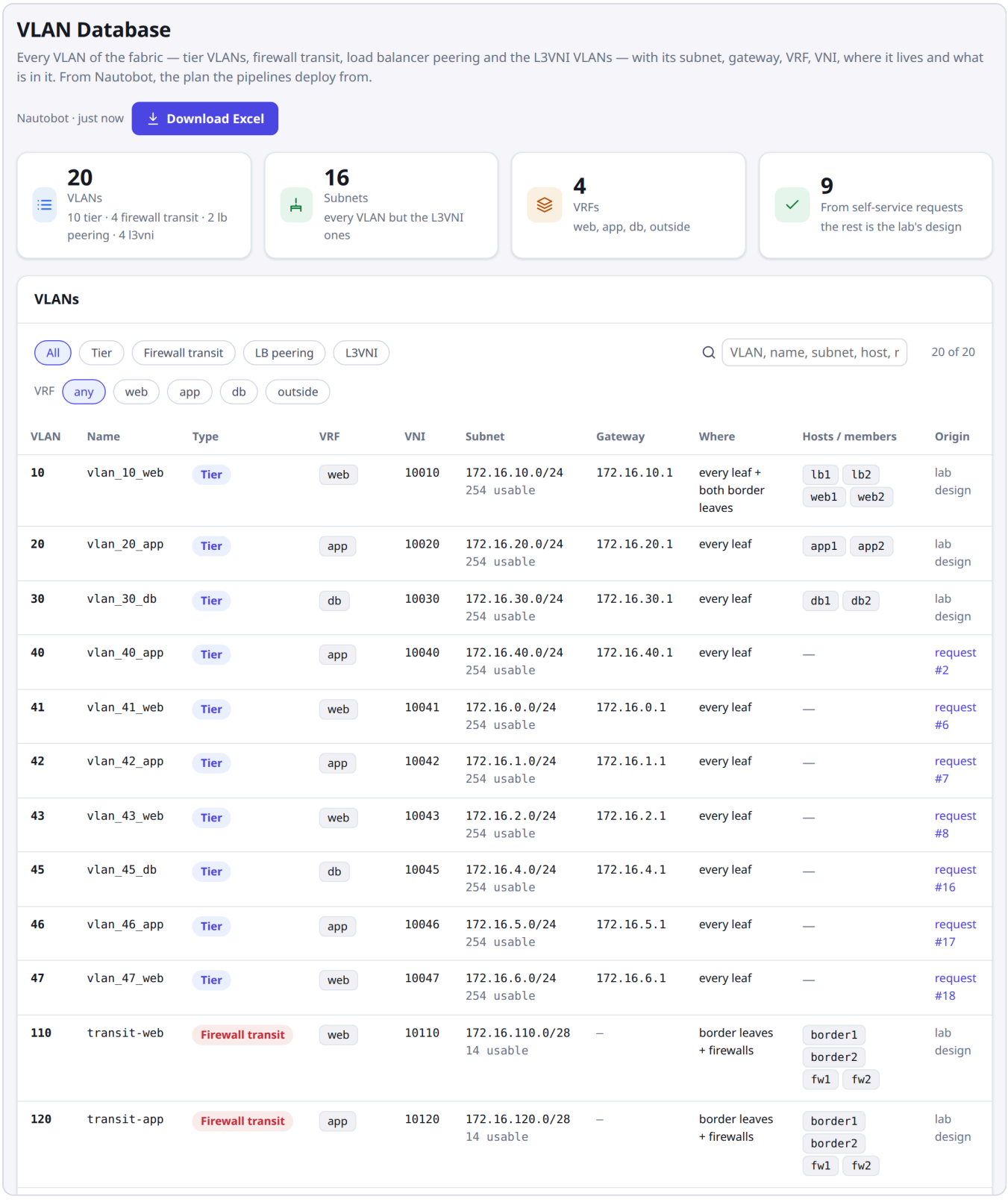


Figure 17. The ASN Database: every AS on the fabric with its devices, router IDs, peers and VRFs; filter by kind or search, and download it as Excel.



VLAN Database

Every VLAN of the fabric — tier VLANs, firewall transit, load balancer peering and the L3VNI VLANs — with its subnet, gateway, VRF, VNI, where it lives and what is in it. From Nautobot, the plan the pipelines deploy from.

Nautobot · just now — Download Excel

20 VLANs — 10 tier · 4 firewall transit · 2 lb peering · 4 l3vni

16 Subnets — every VLAN but the L3VNI ones

4 VRFs — web, app, db, outside

9 From self-service requests — the rest is the lab's design

VLANs

All | Tier | Firewall transit | LB peering | L3VNI

VLAN, name, subnet, host, r — 20 of 20

VRF: any | web | app | db | outside

VLAN	Name	Type	VRF	VNI	Subnet	Gateway	Where	Hosts / members	Origin
10	vlan_10_web	Tier	web	10010	172.16.10.0/24 254 usable	172.16.10.1	every leaf + both border leaves	lb1 lb2 web1 web2	lab design
20	vlan_20_app	Tier	app	10020	172.16.20.0/24 254 usable	172.16.20.1	every leaf	app1 app2	lab design
30	vlan_30_db	Tier	db	10030	172.16.30.0/24 254 usable	172.16.30.1	every leaf	db1 db2	lab design
40	vlan_40_app	Tier	app	10040	172.16.40.0/24 254 usable	172.16.40.1	every leaf	—	request #2
41	vlan_41_web	Tier	web	10041	172.16.0.0/24 254 usable	172.16.0.1	every leaf	—	request #6
42	vlan_42_app	Tier	app	10042	172.16.1.0/24 254 usable	172.16.1.1	every leaf	—	request #7
43	vlan_43_web	Tier	web	10043	172.16.2.0/24 254 usable	172.16.2.1	every leaf	—	request #8
45	vlan_45_db	Tier	db	10045	172.16.4.0/24 254 usable	172.16.4.1	every leaf	—	request #16
46	vlan_46_app	Tier	app	10046	172.16.5.0/24 254 usable	172.16.5.1	every leaf	—	request #17
47	vlan_47_web	Tier	web	10047	172.16.6.0/24 254 usable	172.16.6.1	every leaf	—	request #18
110	transit-web	Firewall transit	web	10110	172.16.110.0/28 14 usable	–	border leaves + firewalls	border1 border2 fw1 fw2	lab design
120	transit-app	Firewall transit	app	10120	172.16.120.0/28 14 usable	–	border leaves + firewalls	border1 border2 fw1 fw2	lab design

Figure 18. The VLAN Database (first rows): each VLAN's subnet, gateway, VRF and VNI, where it lives, its hosts, and the request that made it.

Port Provisioning

A change workflow, approved by an operator like every other: enable or disable a leaf port, and set a spare port up as an access port (one VLAN, untagged) or a trunk (the VLANs chosen, tagged). Every leaf has two spare ports, eth6 and eth7, added for this: unwired, and disabled in Nautobot until someone provisions them.

Port	What can change	Guard
Uplinks to the spines (eth1, eth2)	Nothing	**Blocked.** Listed as such in the form; refused by the form and again by the worker. They carry the fabric.
A server's bond leg (eth3–eth5)	Enable or disable	Disabled only while the server's other leg is up — in Nautobot and live on the partner leaf — so the server is never cut off. Its VLAN follows the server (bond0.<VLAN>): mode and VLANs cannot change here; Move a server does that.
A spare port (eth6, eth7)	Enable or disable; access (one VLAN), trunk (VLANs), or no VLANs	Only VLANs the leaves carry; an enabled port needs a mode; a VLAN provisioned on a port cannot be removed.

The intent lives in Nautobot's native interface fields — enabled, mode (access or tagged), untagged VLAN, tagged VLANs, description — so it is visible and editable where every other part of the plan is. The switches pipeline renders it and its managed deletions take away whatever no longer applies; the request then reads the port's admin state from VyOS and its VLANs from the kernel bridge, and runs the ping test from the WAN. verify knows a leg disabled on purpose is not a fault.

What the switches pipeline renders for leaf1 eth6 as a trunk (VLANs 41 and 47), and for a disabled port:

```
set interfaces ethernet eth6 description 'backup appliance'
set interfaces bridge br0 member interface eth6 allowed-vlan 41
set interfaces bridge br0 member interface eth6 allowed-vlan 47
# access instead:   set interfaces bridge br0 member interface eth6 native-vlan 42
# disabled:         set interfaces ethernet eth6 disable
```

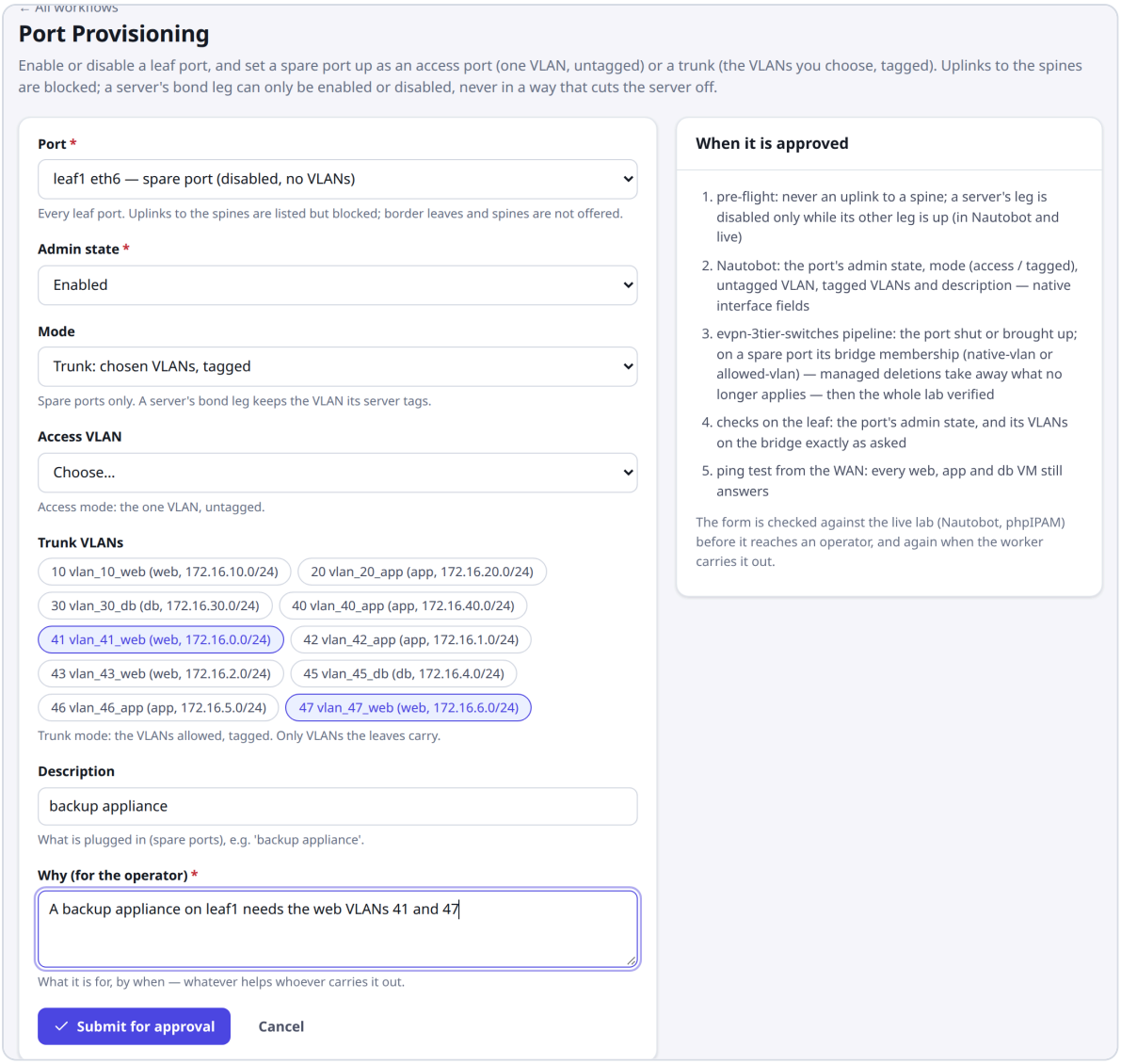


Figure 19. The Port Provisioning form: a spare port as a trunk, its VLANs chosen as chips, and what happens once it is approved.

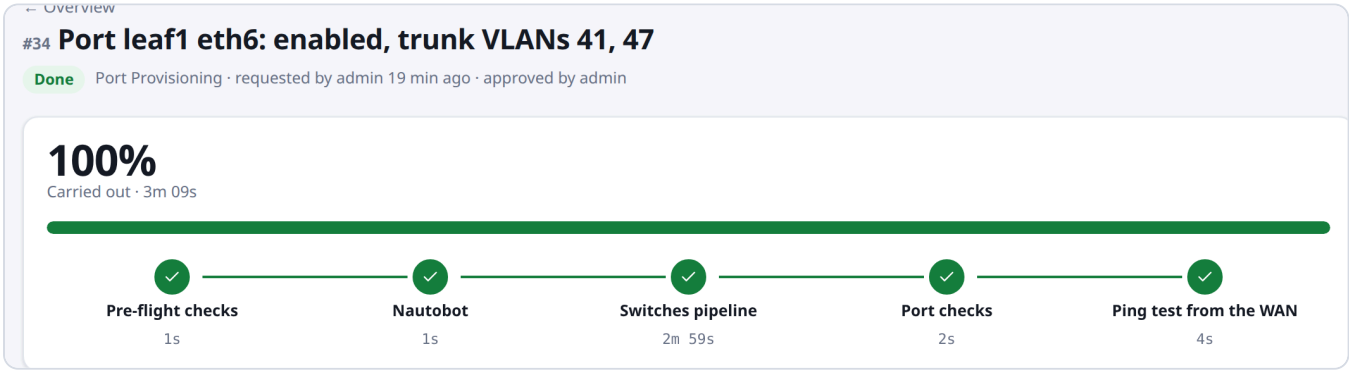


Figure 20. Request #34, done in about three minutes: pre-flight, Nautobot, the switches pipeline (with verify), the port checks and the ping test.

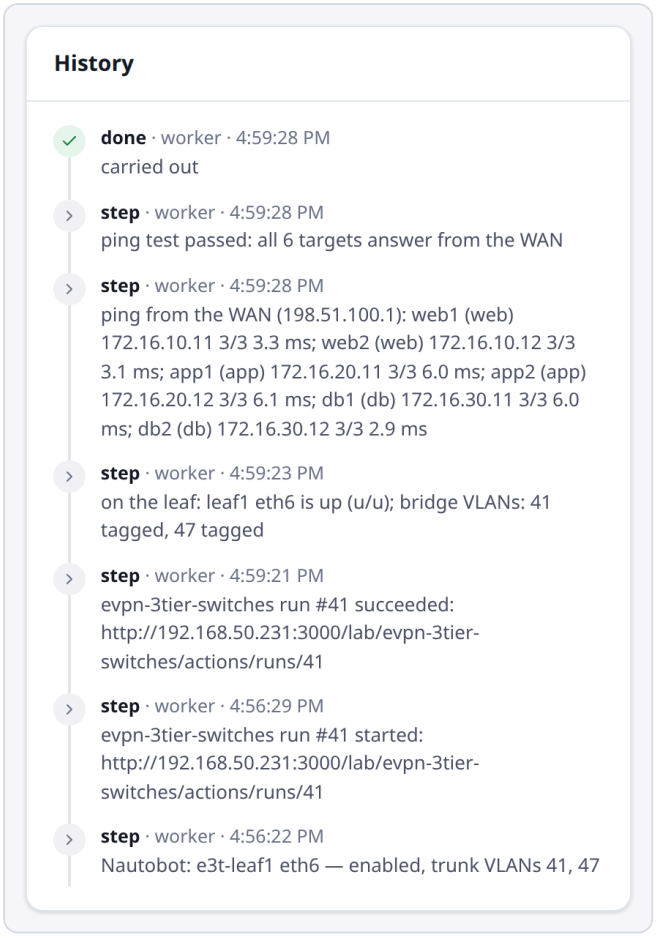


Figure 21. Its history: the leaf's bridge carries exactly VLANs 41 and 47, tagged, and every server still answers from the WAN.

Tested live. Requests #34 to #38 made leaf1 eth6 a trunk (VLANs 41 and 47), then an access port on VLAN 42, then disabled it with no VLANs; disabled web1's leg on leaf1 — web1 ran on its leaf2 leg and the pipeline's verify passed — and enabled it again. A change to an uplink was refused. The first attempt to disable the leg was refused by its own live check, which read the wrong line of VyOS's bond output: it failed safe, changing nothing, and was fixed.

Validation and allocation

Forms are generated from server-side definitions and validated against a live model of the lab built from Nautobot (falling back to lab.conf): VLAN IDs, VNIs, ASNs and overlapping subnets already in use are refused, including those held by other open requests. The VLAN allocator picks the lowest unused ID above the highest tier VLAN, skipping anything in Nautobot, phpIPAM or an open request, and asks phpIPAM for its first free /24 under 172.16.0.0/16 that overlaps nothing. Values are re-checked at submission and again at execution.

Architecture

- **Front end:** React 19 and TypeScript, built with Vite into static files that Django serves from the same origin as the API, so login is one session cookie with no CORS.
- **API:** Django 6.1 with Django Ninja, documented with OpenAPI at /api/docs; served by gunicorn with uvicorn workers (ASGI) for the long-lived event stream.
- **Worker:** a Django management command running as its own service; it carries out one approved request at a time and marks a request interrupted by a restart as failed (retryable).
- **Live updates:** /api/stream sends a server-sent event for each request that changed in the last second; the browser re-fetches what it shows. No message broker is needed.

API	Purpose
GET/POST/DELETE /api/session	Who is signed in (with the CSRF token); sign in; sign out

API	Purpose
GET /api/catalog, /api/catalog/{slug}/form	Workflows and their form definitions
GET /api/catalog/add-vlan/allocate	Next unused VLAN and a free /24 from phpIPAM
GET/POST /api/requests, GET /api/requests/{id}	List by scope (mine, all, pending); create; detail with history and progress
POST /api/requests/{id}/{action}	approve, reject, retry, cancel
GET /api/requests/{id}/pipelines/…/log	A pipeline step's whole log, read live from Gitea
GET /api/devices, …/config, …/state, /api/devices-configs.zip	Device list; configuration; live state by category; a zip of configurations
GET /api/fabric, /api/fabric/health	Fabric Dashboard: the design and its KPIs from Nautobot; live health compared with it
GET /api/tools/asns, /api/tools/asns.xlsx	ASN Database: every ASN and BGP session, as JSON or an Excel workbook
GET /api/tools/vlans, /api/tools/vlans.xlsx	VLAN Database: every VLAN, as JSON or an Excel workbook
GET /api/stream	Server-sent events: requests that changed

Security model

Django accounts in two groups: requesters submit and see their own requests; operators see all, and approve, reject and retry. Every change through the API requires the session's CSRF token. The device tools and the Fabric Dashboard run only commands from a fixed server-side list (the browser names a device and categories, never a command), redact password hashes, keys and similar secrets before anything is shown or downloaded, and log every retrieval with who asked.

9 A change end to end: Add a VLAN

Adding a VLAN touches seven systems and takes about six minutes, almost all of it in the two pipeline runs.

Add a VLAN: reserve in phpIPAM, plan in Nautobot, build with the pipelines, prove from the WAN

Sequence of one Add a VLAN request (about 6 minutes, most of it in the two pipeline runs)

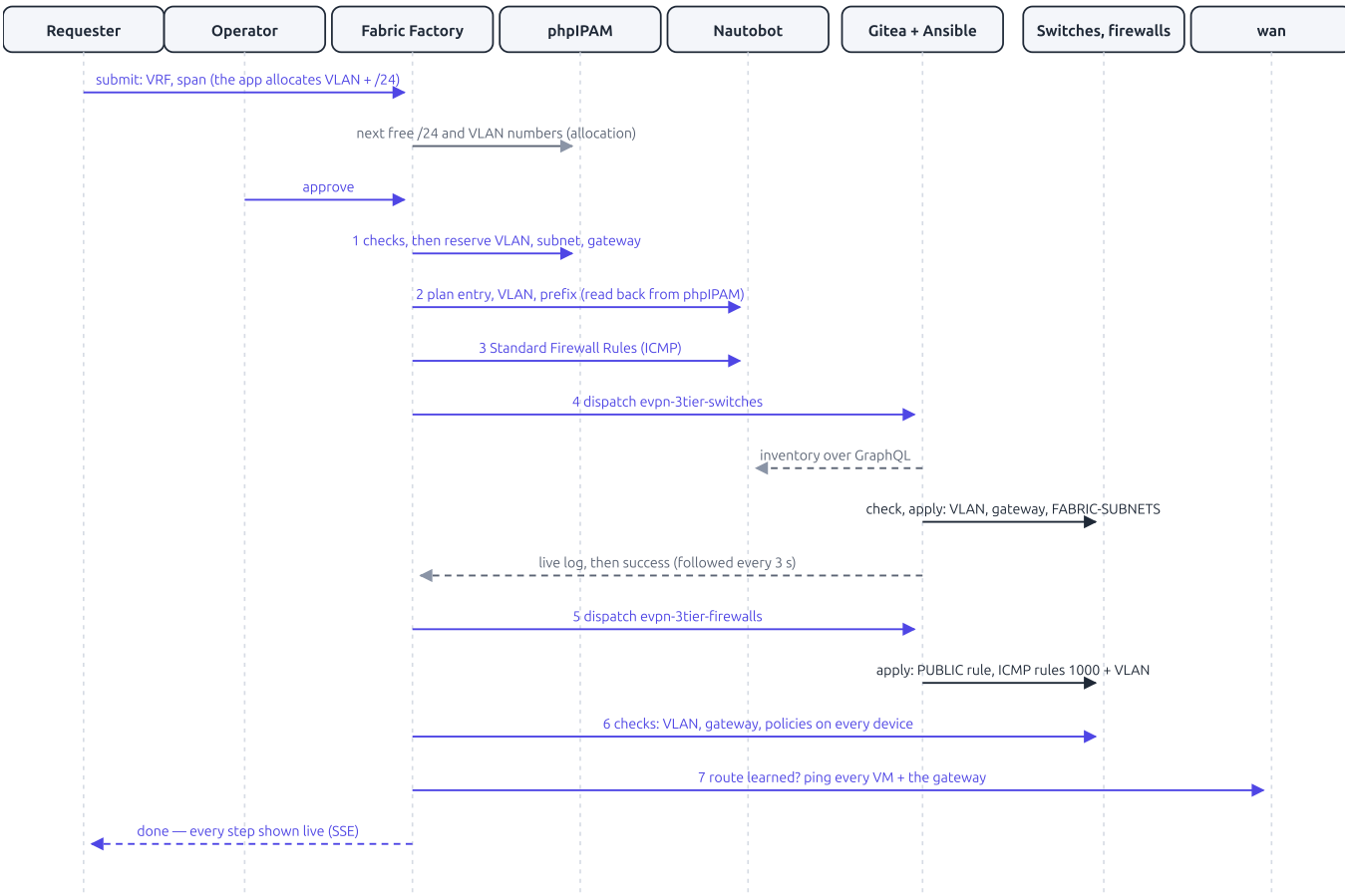


Figure 22. Sequence of an Add a VLAN request. Solid: requests and writes; dashed: reads and observation.

1. **Request.** The user picks a VRF and presses *Select the next unused VLAN ID*; nothing else needs typing except a reason. The app fills in, for example, VLAN 45, VNI 10045, vlan_45_app, 172.16.4.0/24 with gateway 172.16.4.1. An operator approves.
2. **phpIPAM.** Checks first (the VLAN and subnet still free, or already reserved by this request), then the VLAN, the subnet and the gateway address are reserved.
3. **Nautobot.** From what phpIPAM holds, read back: the VLAN enters the plan; the VLAN and prefix objects are created in the VRF.
4. **Standard Firewall Rules.** ICMP to the new subnet from the other zones is added to the firewall policy.
5. **Switches pipeline.** VLAN-to-VNI, the anycast gateway, the VLAN on VXLAN, its EVPN VNI, and the subnet in the borders' FABRIC-SUBNETS.
6. **Firewalls pipeline.** The subnet in PUBLIC (so the WAN learns it) and the new ICMP rules.
7. **Checks.** On every switch in the span, both borders, both firewalls, and the WAN's routing table.
8. **Ping test.** From the WAN's client address, every web, app and db server and the new gateway must answer at least 2 of 3 pings.

Remove a VLAN mirrors it with phpIPAM last: pre-flight checks; Nautobot (plan entry, prefix, VLAN object, its Standard Firewall Rules); the switches and firewalls pipelines, whose managed deletions take every line away; checks that no trace remains on any switch or firewall and the WAN has dropped the route; then phpIPAM releases the subnet, the gateway and the VLAN. Requests #14 and #15 added and removed VLAN 45 end to end, with all seven ping targets answering in between.

10 Resilience and verification

Every single failure is survived, most in a tenth of a second, measured under live load.

Each scenario runs while the WAN's clients send an HTTPS read every 100 ms and an order every second, each on a fresh connection through the firewall to the VIP; one component is powered off for 20 s and then restored.

Component powered off	What takes over	Lost while down	Longest gap	After restore
lb1	keepalived moves the VIP to lb2	0–4 in flight	0.1 s	0 lost
web1	HAProxy health check and redispatch	0–1 in flight	0.1 s	0 lost
app1	nginx uses the other app server	0	0.1 s	0 lost
fw1	BFD + BGP move every VRF to fw2; conntrack-sync keeps sessions	0	0.1 s	0 lost
leaf1	EVPN multihoming: each server's other leg	0	0.1 s	0 lost
border1	border2 for firewalls, load balancers and WAN	4 reads	0.3 s	0 lost
db1 (db2 promoted after 5 s)	The app finds the new primary itself	32 reads, 3 writes	3.3 s	switched back

verify runs 65 checks: BGP and BFD sessions, EVPN and Ethernet segments, the firewalls' sessions, preference and VRRP, the data plane, the application end to end (an order written from the WAN reaches every replica), and the policy matrix, which opens temporary listeners on the targets and tries every TCP flow from every tier and the WAN (35 flows from the WAN, exactly 2 allowed). The pipelines run its quick mode (61 checks) after every apply.

Nine Robot Framework suites (fabric, firewalls, data plane, application, policy, failures, network, database, firewall sessions) assert all of the above including the failure scenarios; a full run takes about 35 minutes, the non-disruptive subset about 3.

11 The tech stack

Entirely open source; versions as deployed on 9 November 2026.

Layer	Component	Version	Runs on	Role and interfaces
Compute	Ubuntu Server	26.04.1 LTS, kernel 7.0	Lab host	12 cores, 57 GB RAM; runs every VM and the lab services
Compute	KVM / QEMU	10.2.1	Lab host	Virtual machines; UDP tunnels as fabric cables
Compute	libvirt	12.0.0	Lab host	VM and management-network lifecycle (virsh)
Network OS	VyOS (rolling)	2026.09.09	11 VMs	Spines, leaves, borders, firewalls, WAN; config CLI, op mode
Routing	FRRouting	10.6.1	In VyOS	BGP (unnumbered, EVPN), BFD, VRFs
Firewall	VyOS zone firewall, conntrackd	in VyOS	fw1, fw2	Zone policy, connection-state sync, VRRP
Server OS	Alpine Linux	3.22.4	9 VMs	Small images; OpenRC services
Load balancing	HAProxy	3.2.26	lb1, lb2	TLS termination, round robin, health checks, stats
Load balancing	keepalived	2.3.1	lb1, lb2	VRRP for the web VIP (virtual MAC)
Load balancing	BIRD	2.17.2	lb1, lb2	eBGP to the borders, VIP prefixes, BFD
Web	nginx	1.28.3	web1, web2	Page, health, reverse proxy to the app tier
Application	Python, psycopg	3.12.13, psycopg 3	app1, app2	The Order Desk API on port 8080
Database	PostgreSQL	17.11	db1, db2	Primary and streaming hot standby
IPAM	phpIPAM, MariaDB, Docker	1.8.3, 11.8.8	IPAM VM	Addressing source of truth; REST API
Source of truth	Nautobot (PostgreSQL 16, Redis 7, Celery)	3.2.4	NMS (Docker)	Plan, devices, cabling; REST and GraphQL
CI/CD	Gitea, Gitea Actions runner	1.24.7, runner 3.5.0	NMS; runner on the host	Repositories, pipelines, run history
Automation	ansible-core	2.20.11	Runner venv	Playbooks, Jinja templates, check mode
Automation	vyos.vyos, ansible.netcommon, ansible.utils, community.general	vyos.vyos 6.2.0	Runner venv	VyOS modules, network_cli, filters, doas

Layer	Component	Version	Runs on	Role and interfaces
Self-service	Django, Django Ninja	6.1.3, 1.8.0	Lab host	Requests, approval, worker, JSON API
Self-service	gunicorn, uvicorn	26.2.0, 0.54.0	Lab host	ASGI server for the event stream
Self-service	SQLite	with Python	Lab host	Requests, history, progress snapshots
Self-service	React, TypeScript, Vite, React Router	19.2.0, 5.9.3, 7.2.2, 7.9.5	Browser	The Fabric Factory front end
Build	Node.js	24.11.1 LTS	Lab host	Builds the front end only
SSH tooling	paramiko	4.0.0	Lab host	Device tools, checks, ping test
Monitoring	Prometheus, Grafana	3.14.0, 13.0.2	NMS (Docker)	Metrics, alerts, the evpn-3tier dashboard
Testing	Robot Framework	7.5.1	Lab host	Nine test suites

12 Operating the lab

One script builds and tests the lab; Fabric Factory changes it; a portal, the Lab Hub and Grafana watch it.

Command	What it does
lab.sh gen / up / bootstrap / configure	Render configurations from lab.conf; create and start the VMs; day-0 configuration over the serial consoles; re-apply over SSH
lab.sh check-wiring	Prove every link reaches the intended peer port (MAC address, LACP partner)
lab.sh services	Deploy the application: PostgreSQL primary and replica, the app, nginx, HAProxy and keepalived
lab.sh verify [--quick]	The 65 (61) end-to-end checks
lab.sh chaos <component>, fw-failover	Failure scenarios under load; a live connection through a firewall failover
lab.sh test	The Robot Framework suites
lab.sh db status | promote | reseed | switchover	Database roles
lab.sh status, show <node> <cmd>, ssh <node>, down	Inventory, operational commands, shell access, shutdown

Monitoring. Prometheus scrapes the portal's metrics and raises alerts for the application, nodes, BGP sessions, multihoming legs, services, the active firewall, the database roles and failed tests (silenced while a portal run is in progress). Grafana shows the overview dashboard. The portal shows, every 5 s, which firewall, load balancer, web, app and database server a real request took.

Change path. In daily operation, changes go through Fabric Factory and the pipelines. lab.sh rebuilds the lab from its own description and runs tests; VLANs and peerings added through Fabric Factory live in Nautobot, which the pipelines report as drift against lab.conf.

13 Design decisions and lessons learned

Most of the design follows from failures found while building the lab; each is now guarded by a check or a test.

Decision	Why
A VRF per tier, firewalls as the only path	Isolation enforced by routing: a tier has no route anywhere except through a firewall
No /32s or transit addresses to the firewalls (TO-FW)	EVPN placed the other border's transit address in the VRF as a /32; forwarded, it sent the firewalls' BFD through the wrong border and the sessions died
as-override on the firewalls' outside sessions	One border dropped the web route on the way back (its own AS in the path); with the other border down the WAN had no route
Segment bonds boot disabled, raised after EVPN converges	A rebooting leaf otherwise attracted traffic about 6 s before it could forward it
A virtual MAC for the web VIP	With the holder's own MAC, the failover's gratuitous ARP reached one border only; half the flows went to a dead load balancer for minutes
nginx evicts app servers only on connection errors and timeouts	Counting the app's database-down answers turned a 3 s leaf failover into 13 s of errors
phpIPAM first when adding, last when removing	Addresses are reserved before anything uses them and freed only after nothing does
Stable rule numbers keyed to VLAN IDs	Adding or removing a VLAN never renumbers other rules, keeping deletions small and reviewable
Inventory committed on every run	Each run's exact input is versioned and comparable with the next
ansible-core pinned to 2.20	2.21 removed the paramiko connection plugin the VyOS modules use on this host
Device tools with a fixed command list and redaction	Read-only visibility for every user without a path to arbitrary commands
The worker follows Gitea's web job endpoint	Gitea 1.24 has no API for a running job's log; without it, a pipeline run is minutes of silence

14 Limitations and roadmap

The lab is a working model, not a production design; these gaps are known.

- **phpIPAM and Nautobot are not reconciled.** The workflows write both in a fixed order, but nothing repairs a difference made by hand in either system.
- **lab.conf lags the plan.** VLANs and peerings added through Fabric Factory live only in Nautobot, so the lab's own renderer and the pipelines disagree about them (reported as drift).
- **Load balancer peering writes phpIPAM last**, unlike Add a VLAN.
- **Firewall fail-back and idle connections.** After a fail-back the standby learns only connections whose state changes, so a long-idle connection can be missing on it.
- **Scale.** One host; the self-service app uses SQLite and one worker; pipelines run one at a time.
- **Lab-grade security.** Self-signed certificates, plain HTTP on the home network and shared lab logins.
- **Roadmap.** Layer-4 load-balancing experiments; reconciliation between phpIPAM and Nautobot; more self-service workflows.